



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 1

Aim: To Implement Loading the data from CSV or JSON files using SQL command (PostgreSQL) or LOAD DATA (MySQL).

Prerequisite: PostgreSQL, MySQL and SQL Commands.

Theory: Fundamentals with Introduction to Data Engineering, Data Extraction, Transformation, and Loading.

Introduction to Data Engineering

Data engineering is the process of designing, building, and maintaining systems that allow organizations to collect, store, process, and analyze large volumes of data. It involves a range of activities such as data pipeline creation, database management, data integration, and optimization to ensure that data is available and accessible for analysis. Data engineers play a crucial role in transforming raw data into a format that is suitable for analysis by data scientists, business analysts, and decision-makers. The field combines knowledge of programming, databases, cloud services, and distributed systems to handle data at scale.

Data Extraction

Data extraction is the first step in the process of obtaining valuable insights from various data sources. It involves retrieving data from different systems, formats, or storage platforms, such as databases, APIs, files, or web scraping. The goal is to extract raw data in a structured or unstructured form, making it ready for processing. Techniques for data extraction vary depending on the source and can involve SQL queries, APIs, or specialized tools like ETL (Extract, Transform, Load) software. The quality and completeness of the extracted data are essential for accurate analysis and decision-making.

Data Transformation

Data transformation is the process of converting raw data into a more usable and meaningful format for analysis. It involves cleaning, filtering, and modifying data to ensure it is consistent, accurate, and structured according to the needs of the end-user or system. This may include tasks like normalizing values, aggregating data, changing data types, handling missing values, and creating new calculated columns. Data transformation is typically performed as part of an ETL process and is crucial for ensuring that data can be effectively analyzed and integrated into business processes.

Data Loading

Data loading is the final step in the ETL (Extract, Transform, Load) pipeline, where the transformed data is loaded into a destination system, such as a data warehouse, database, or analytics platform. This step ensures that the data is stored in a format optimized for querying, reporting, and analysis. The loading process can be either batch-based (loading data in large chunks at scheduled intervals) or real-time (loading data continuously as it becomes available). The choice of data loading method depends on the business needs, the volume of data, and the frequency of updates required. Efficient data loading is key to ensuring timely and accurate access to data for decision-makers.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Implementation Steps:

1) Install PostgreSQL

1. PostgreSQL: Loading Data Using SQL Commands

For CSV Files:

You can use the COPY command or the \copy meta-command in PostgreSQL.

Steps:

1. Ensure the CSV file is accessible to the PostgreSQL server.
2. The table structure in the database must match the structure of the CSV file.
3. Use the COPY command as follows:

Example of loading data into a table named 'users'

```
COPY users (id, name, email, age)
FROM '/path/to/your/file.csv'
DELIMITER ','
CSV HEADER;
```

Explanation:

- users: The table where data will be loaded.
- (id, name, email, age): Column names corresponding to the CSV file columns.
- /path/to/your/file.csv: Full path to the CSV file.
- DELIMITER ',': Defines the delimiter in the CSV file (default is a comma).
- CSV HEADER: Indicates the CSV file has a header row.

Alternative: Using \copy in psql:

If you don't have access to the server filesystem, use the \copy command, which works on the client side.

```
\copy users (id, name, email, age)
```



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

```
FROM '/path/to/your/file.csv'  
DELIMITER ','  
CSV HEADER;
```

For JSON Files:

PostgreSQL has built-in support for JSON data using the json or jsonb data types. However, you may need to preprocess the JSON file or load it using COPY combined with a temporary table.

1. Create a table with a jsonb column:

```
CREATE TABLE json_data (  
    id SERIAL PRIMARY KEY,  
    data jsonb  
);
```

2. Use the COPY command:

```
COPY json_data(data)  
FROM '/path/to/your/file.json';
```

3. You can then extract or transform the JSON data using PostgreSQL's JSON functions.
-

2. MySQL: Loading Data Using LOAD DATA

For CSV Files:

MySQL supports the LOAD DATA INFILE command to load CSV data efficiently.

Steps:

1. Ensure the CSV file is accessible to the MySQL server.
2. The table structure must match the structure of the CSV file.
3. Use the following command:

```
LOAD DATA INFILE '/path/to/your/file.csv'  
INTO TABLE users  
FIELDS TERMINATED BY ','  
ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS  
(id, name, email, age);
```

Explanation:

- LOAD DATA INFILE: Loads the data from a file.
- /path/to/your/file.csv: Full path to the CSV file.
- FIELDS TERMINATED BY ',': Specifies the delimiter in the file.
- ENCLOSED BY '"': Specifies the enclosing character for fields (optional).
- LINES TERMINATED BY '\n': Specifies the line-ending character.
- IGNORE 1 ROWS: Skips the header row.
- (id, name, email, age): Maps the file columns to table columns.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Note: If the file is not accessible on the server, you can use LOAD DATA LOCAL INFILE instead:

```
LOAD DATA LOCAL INFILE '/path/to/your/file.csv'  
INTO TABLE users  
FIELDS TERMINATED BY ','  
ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS;
```

For JSON Files:

MySQL does not natively support loading JSON files into a table like it does for CSV files. You can:

1. Parse the JSON file in your application (e.g., using Python or JavaScript) and insert data into the table using INSERT statements.
2. Alternatively, use MySQL's JSON data type and insert raw JSON.

Example:

```
CREATE TABLE json_data (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  data JSON  
);
```

-- Insert JSON data directly

```
INSERT INTO json_data (data)
```

```
VALUES ('{"name": "John", "email": "john@example.com", "age": 30}');
```

For bulk insertion, preprocess the JSON file into a CSV format or use a custom script.

OUTPUT SCREENSHOTS:

FOR CSV

1. CREATE DATABASE

```
1 • CREATE DATABASE heart_attack_db;
```



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

CREATE TABLE

SQL Query Editor

```
CREATE TABLE heart_attack (  
    id SERIAL PRIMARY KEY,  
    age INT,  
    sex INT,  
    total_cholesterol FLOAT,  
    ldl FLOAT,  
    hdl FLOAT,  
    systolic_bp FLOAT,  
    diastolic_bp FLOAT,  
    smoking INT,  
    diabetes INT,  
    heart_attack INT  
);
```

CREATE TABLE

Query returned successfully in 128 msec.

2. LOAD CSV FILE

```
COPY heart_attack(age, sex, total_cholesterol, ldl, hdl, systolic_bp, diastolic_bp, smc  
FROM 'C:\Program Files\PostgreSQL\15\data\updated_version.csv'  
DELIMITER ','  
CSV HEADER;
```

COPY 1000

Query returned successfully in 78 msec.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

3. DISPLAY TABLE

```
SELECT * FROM heart_attack;
```

Data Output Messages Notifications											
	id [PK] Integer	age Integer	sex Integer	total_cholesterol double precision	ldl double precision	hdl double precision	systolic_bp double precision	diastolic_bp double precision	smoking Integer	diabetes Integer	heart_attack Integer
1	1	57	1	229.46364245989878	175.87912925154933	39.22568705527789	124.07012716313945	91.37878025428563	0	0	0
2	2	58	1	186.46411960718476	128.9849160885468	34.95096762813148	95.49255176716945	64.35504019073372	1	0	0
3	3	37	1	251.30071919021645	152.34759243501477	45.91328769795121	99.51933468568974	64.95314701865036	0	1	0
4	4	55	1	192.05890823360733	116.80368391178928	67.20892502595552	122.46000218901882	73.8213817518664	0	0	0
5	5	53	1	151.20344845928565	107.01739560621428	60.69383790566267	123.02225716752147	81.12194576039188	0	1	0
6	6	39	1	236.0334551699892	153.88080937296337	31.208614010237696	121.85739560481021	79.58906898043419	0	0	0
7	7	65	0	174.6156654741166	114.02940746386828	55.69258599001828	135.60504971915043	85.52995490168766	0	0	0
8	8	33	0	242.91940159950704	147.95137529846954	54.4394748110479	123.51155667862578	77.33171433136184	0	0	0
9	9	49	0	95.80435914010542	83.30487485574838	60.758928918985845	111.69748795963216	77.63052930816697	1	0	0
10	10	55	0	181.36094256462488	106.01178251788478	50.57674663030641	129.57641759900724	87.58878129615607	0	0	0
11	11	51	0	172.48884163155563	108.37382246121102	51.2036260423836	96.82124634895237	69.57381612195707	0	0	0
12	12	46	1	182.77891098453784	113.6543939042498	46.83824528078549	114.31364253763297	79.97690932376277	0	0	0
13	13	54	1	126.9359199405608	88.47328560934073	59.03780511574431	151.13849059330406	96.92753428227385	0	0	0
14	14	33	0	219.195145421094	149.29948851137334	39.257935850175556	135.85762085722814	81.29240087203232	0	0	0
15	15	35	1	196.43572789632594	115.05486999260103	40.61167128512642	113.22721541605009	71.2237534490105	0	0	0
16	16	73	1	200.40970895710234	151.5826471964596	35.832455981749455	129.9974758740287	93.53521042451113	1	0	1
17	17	54	0	152.98228660589035	80.54516819341904	63.81022315315402	119.63183529642467	81.36480595501722	0	0	0
18	18	53	0	173.63153909414024	148.25682007086246	50.798153909782016	108.55837553210901	77.91457649513478	0	0	0
19	19	67	1	247.6771509726762	170.19838132056907	48.25316658394561	156.19553877301414	96.90645422915483	0	0	0
20	20	38	1	206.90306810921524	135.2881048032097	55.74446873727193	119.28299242337216	87.95509922514194	0	0	0
21	21	59	1	163.73310940575246	120.58903417798795	50.280963120194926	131.69316724202164	89.72471207901188	0	0	0
22	22	28	1	221.7970108896927	139.90417877425313	38.288142247820076	116.22689116706718	81.21362757245257	1	0	0
Total rows: 1000 of 1000 Query complete 00:00:00.297											



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

FOR JSON

1. CREATE TABLE

```
1 CREATE TABLE digital_literacy (  
2     user_id TEXT PRIMARY KEY,  
3     age INT,  
4     gender TEXT,  
5     education_level TEXT,  
6     employment_status TEXT,  
7     household_income TEXT,  
8     location_type TEXT,  
9     basic_computer_knowledge_score INT,  
10    internet_usage_score INT,  
11    mobile_literacy_score INT,  
12    post_training_basic_computer_knowledge_scc  
13    post_training_internet_usage_score INT,  
14    post_training_mobile_literacy_score INT,  
15    modules_completed INT,  
16    average_time_per_module FLOAT,  
17    quiz_performance INT,  
18    session_count INT,  
19    engagement_level TEXT,  
20    adaptability_score INT,  
21    feedback_rating INT,  
22    skill_application INT,  
23    employment_impact TEXT,  
24    overall_literacy_score FLOAT  
25 );  
26
```

CREATE TABLE

Query returned successfully in 76 msec.

2. INSERT JSON FILE INTO SQL USING PYTHON SCRIPT



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

```
import json.py > ...
1 import json
2 import psycopg2
3
4 # Load JSON file
5 file_path = "F:\digital_literacy_dataset.json" # Adjust this path if needed
6 with open(file_path, "r") as f:
7     data = json.load(f)
8
9 # Connect to PostgreSQL
10 conn = psycopg2.connect(
11     dbname="...",
12     user="...",
13     password="...",
14     host="localhost",
15     port="5432"
16 )
17 cur = conn.cursor()
18
19 # Insert data into the table
20 for row in data:
21     cur.execute("""
22         INSERT INTO digital_literacy (
23             user_id, age, gender, education_level, employment_status, household_income,
24             location_type, basic_computer_knowledge_score, internet_usage_score,
25             mobile_literacy_score, post_training_basic_computer_knowledge_score,
26             post_training_internet_usage_score, post_training_mobile_literacy_score,
27             modules_completed, average_time_per_module, quiz_performance, session_count,
28             engagement_level, adaptability_score, feedback_rating, skill_application,
29             employment_impact, overall_literacy_score
30         ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
31     """, (
32         row["User_ID"], row["Age"], row["Gender"], row["Education_Level"], row["Employment_Status"],
33         row["Household_Income"], row["Location_Type"], row["Basic_Computer_Knowledge_Score"],
```

```
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
34     """, (
35         row["User_ID"], row["Age"], row["Gender"], row["Education_Level"], row["Employment_Status"],
36         row["Household_Income"], row["Location_Type"], row["Basic_Computer_Knowledge_Score"],
37         row["Internet_Usage_Score"], row["Mobile_Literacy_Score"], row["Post_Training_Basic_Computer_Knowledge_Score"],
38         row["Post_Training_Internet_Usage_Score"], row["Post_Training_Mobile_Literacy_Score"],
39         row["Modules_Completed"], row["Average_Time_Per_Module"], row["Quiz_Performance"],
40         row["Session_Count"], row["Engagement_Level"], row["Adaptability_Score"], row["Feedback_Rating"],
41         row["Skill_Application"], row["Employment_Impact"], row["Overall_Literacy_Score"]
42     ))
43
44 # Commit and close connection
45 conn.commit()
46 cur.close()
47 conn.close()
48
49 print("Data imported successfully!")
```



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

3. DISPLAY TABLE

```
SELECT * FROM digital_literacy LIMIT 10;
```

	user_id [PK] text	age integer	gender text	education_level text	employment_status text	household_income text	location_type text	basic_computer_knowledge_score integer	internet_usage_score integer	mobile_literacy_score integer	post_training_basic_co integer
1	U0001	43	Male	Primary	Student	Medium	Semi-Rural	25	1	33	
2	U0002	60	Female	High School	Farmer	Low	Rural	22	14	35	
3	U0003	47	Female	Primary	Farmer	Low	Semi-Rural	14	31	14	
4	U0004	34	Female	Secondary	Farmer	Low	Rural	6	32	17	
5	U0005	50	Male	High School	Other	Medium	Rural	14	41	19	
6	U0006	57	Female	High School	Self-Employed	Low	Rural	15	3	46	
7	U0007	46	Other	Secondary	Unemployed	Low	Semi-Rural	35	34	49	
8	U0008	36	Male	High School	Unemployed	Low	Rural	19	48	46	
9	U0009	64	Male	High School	Other	Low	Rural	33	10	37	
10	U0010	61	Female	High School	Self-Employed	Medium	Rural	50	18	20	

Conclusion

We successfully loaded JSON and CSV data into a PostgreSQL table using `COPY` and `PYTHON`. The process included:

1. **Creating the `digital_literacy` table** with columns matching your JSON structure.
2. **Inserting the data** while ensuring proper UTF-8 encoding (using `utf8mb4`).



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

3. **Handling a character set warning** by switching to `utf8mb4` for better compatibility.
4. **Verifying the data** was correctly inserted by running a `SELECT` query.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 2

Aim: To build an end-to-end ETL pipeline: extract data from an API, transform it and load it into a database.

Theory:

Introduction to ETL

ETL stands for Extract, Transform, Load, which is a process used in data integration and data management. It's typically used to prepare data for analysis or reporting. Here's a breakdown of each step:

1. Extract:
 - This is the first step, where data is pulled from various sources such as databases, APIs, flat files, or cloud services. The goal is to gather raw data from diverse and possibly disparate systems.
2. Transform:
 - Once data is extracted, it undergoes transformation, which involves cleaning, structuring, and converting it into a desired format. This step might involve filtering data, performing calculations, normalizing values, handling missing data, or combining information from multiple sources.
3. Load:
 - The final step is to load the transformed data into a target system, which could be a data warehouse, database, or data lake. The data is now ready for analysis, reporting, or further processing.

ETL is critical for preparing data for business intelligence tools, ensuring that the data is accurate, consistent, and usable for decision-making.

Implementation of ETL pipeline

To implement an ETL pipeline, you typically need to write scripts or use tools designed for data extraction, transformation, and loading. Below is a simple outline of how to implement an ETL pipeline, followed by an example using Python and libraries like pandas for transformation, SQLAlchemy for database interaction, and requests for API data extraction.

Steps to Implement an ETL Pipeline

1. Extract Data:
 - Connect to the data source (API, database, or file).
 - Extract raw data from the source.
2. Transform Data:



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

- Clean the data by handling missing values, duplicates, or irrelevant information.
 - Convert data into a structured format for easier processing (e.g., flattening nested data).
 - Perform any calculations or data aggregation as needed.
3. Load Data:
- Load the transformed data into a target system (e.g., a database or data warehouse).

Screenshots:

1) Connection Establishment:

```
In [1]: pip install mysql-connector-python

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: mysql-connector-python in c:\users\djsce.student\appdata\roaming\python\python311\site-packages
(9.2.0)
Note: you may need to restart the kernel to use updated packages.

In [1]: import mysql.connector

mydb = mysql.connector.connect(
    host = 'localhost',
    user = 'root',
    password = 'pass@123',
    database = 'dbase'
)

print(mydb)

<mysql.connector.connection.MySQLConnection object at 0x00000182C9FC6910>
```

2) Retrieving data from database

```
In [2]: mycursor = mydb.cursor()

In [3]: mycursor.execute('SELECT * FROM employee')

In [4]: myresult = mycursor.fetchall()

for x in myresult:
    print(x)

(1, 'Ashok', 'Badlapur', 'BCOM', 101, 'Sales', 'Manager', datetime.date(2000, 9, 22), datetime.date(2023, 8, 11), None)
(2, 'Shravan', 'Nagpur', 'BSC', 102, 'system', 'HOD', datetime.date(1982, 8, 18), datetime.date(2005, 2, 7), None)
(3, 'Adhur', 'Andheri', 'BFM', 103, 'system', 'Senior_Manager', datetime.date(1977, 8, 2), datetime.date(2004, 1, 2), None)
```

3) Extracting data from API



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

EXTRACTING DATA FROM API

```
[10]: pip install requests

Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: requests in c:\programdata\anaconda3\lib\site-packages (2.31.0)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\programdata\anaconda3\lib\site-packages (from requests) (2.0.4)
Requirement already satisfied: idna<4,>=2.5 in c:\programdata\anaconda3\lib\site-packages (from requests) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\programdata\anaconda3\lib\site-packages (from requests) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in c:\programdata\anaconda3\lib\site-packages (from requests) (2024.2.2)
Note: you may need to restart the kernel to use updated packages.

[11]: import requests
import mysql.connector
from datetime import datetime

# 1. Fetch data from the NewsAPI
api_url = 'https://newsapi.org/v2/everything?q=tesla&from=2025-01-04&sortBy=publishedAt&apiKey=b83c45154a7546238ef532c75720999f'
response = requests.get(api_url)

# Check if the request was successful (status code 200)
if response.status_code == 200:
    data = response.json() # Convert the response to a dictionary
else:
    print(f"Error fetching data: {response.status_code}")
    exit()

# 2. Connect to MySQL Database
db = mysql.connector.connect(
    host="localhost", # Database server address
    user="root", # Database username
    password="pass@123", # Database password
    database="dbase" # Database name
)

cursor = db.cursor()

# 3. Create the table (if it doesn't exist already)
cursor.execute("""
CREATE TABLE IF NOT EXISTS XYZ_NEWS (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255),
    description TEXT,
    url VARCHAR(1024),
    published_at DATETIME
)
""")

# 4. Insert data into the MySQL database
articles = data.get('articles', []) # Extract articles list from the response
for article in articles:
    title = article.get('title')
    description = article.get('description')
    url = article.get('url')
    published_at = article.get('publishedAt')

    # Convert the 'publishedAt' value to a MySQL-compatible DATETIME format
    if published_at:
        # Remove the 'Z' at the end and replace 'T' with a space
        published_at = published_at.replace('T', ' ').replace('Z', '')
        # Optionally, you can also convert it to a datetime object
        try:
            published_at = datetime.strptime(published_at, '%Y-%m-%d %H:%M:%S')
        except ValueError:
            print(f"Invalid date format: {published_at}")
            continue

    # Insert data into the table
    cursor.execute("""
INSERT INTO XYZ_NEWS (title, description, url, published_at)
VALUES (%s, %s, %s, %s)
""", (title, description, url, published_at))

# Commit the transaction
db.commit()

# 5. Close the database connection
# cursor.close()
# db.close()

print("Data inserted successfully!")

Data inserted successfully!
```

4) Displaying extracted data



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

DISPLAYING FROM EXTRACTED DATA

```
[14]: mycursor = db.cursor()
mycursor.execute("select * from XYZ_NEWS")
myresult = mycursor.fetchall()

for x in myresult:
    print(x)
```

```
(36, 'USAID turmoil: DOGE reps take over offices, senior officials placed on leave - NewsBreak', 'Amid ongoing turmoil inside the U.S. Agency for International Development, sources say that Department of Government Efficiency staffers have moved to take', 'https://www.newsbreak.com/news/3788205413842-usaid-turmoil-doge-reps-take-over-offices-senior-officials-placed-on-leave?noAds=1&f=app_share&s=a2', datetime.datetime(2025, 2, 3, 0, 56, 53))
(37, 'USAID turmoil: DOGE reps take over offices, senior officials placed on leave - NewsBreak', 'Amid ongoing turmoil inside the U.S. Agency for International Development, sources say that Department of Government Efficiency staffers have moved to take', 'https://www.newsbreak.com/share/3788205413842-usaid-turmoil-doge-reps-take-over-offices-senior-officials-placed-on-leave?_f=app_share&pd=04pDrK4n&lang=en_US&send_time=1738543982&trans_data=%7B%22platform%22%3A%22c%22%3A%2224.39.1.3%22%22languages%22%3A%22en%22%7D&sep=ns_push_exp_rt_bucketv12-v2%2Cns_foryou_blend_exp_25q1-v2%2Cns_foryou_model_exp_25q1-v8%2Cns_foryou_rank_exp_25q1-v4%2Cns_foryou_recall_exp_25q1-v9&s=i2', datetime.datetime(2025, 2, 3, 0, 53, 34))
(38, 'USAID turmoil: DOGE reps take over offices, senior officials placed on leave - NewsBreak', 'Amid ongoing turmoil inside the U.S. Agency for International Development, sources say that Department of Government Efficiency staffers have moved to take', 'https://www.newsbreak.com/share/3788205413842-usaid-turmoil-doge-reps-take-over-offices-senior-officials-placed-on-leave?_f=app_share&pd=0JN8T2pw&lang=en_US&send_time=1738543591&trans_data=%7B%22platform%22%3A%22c%22%3A%2225.4.0.27%22%22languages%22%3A%22en%22%7D&sep=ns_foryou_blend_exp_25q1-v1%2Cns_foryou_rank_exp_25q1-v9%2Cns_foryou_recall_exp_25q1-v9%2Cns_foryou_model_exp_25q1-v6%2Cns_push_exp_rt_bucketv12-v8', datetime.datetime(2025, 2, 3, 0, 47, 3))
(39, 'Singapore says Nvidia's astounding local sales don't mean it's the source of DeepSeek's GPUs', 'Singapore says Nvidia's astounding local sales don't mean it's the source of DeepSeek's GPUs\nPLUS: Chinese bus lanes put Tesla in a tangle; India drops electronics tariffs; Samsung worries about
```

Conclusion:

Thus we successfully built an end-to-end ETL pipeline: extracted data from an API, transformed it and loaded it into a database.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 3

Aim:

1. To load a large dataset into a data warehouse and run queries on it. (e.g. Google or BigQuery).
2. To implement SQL-based transformations using data build tool to build a reliable data transformation online.

Theory:

How to Load a large dataset into a data warehouse and run queries on it. (e.g., Google or BigQuery).

Step 1: Preparing the Dataset

Before loading the dataset into a data warehouse, ensure:

- The dataset is in a supported format (CSV, JSON, Avro, ORC, Parquet).
- The data is **clean** (no missing or corrupted values).
- The schema (columns, data types) is well-defined.

For BigQuery:

- The **file size limit** for a single load job is **5 TB**.
- If using Google Cloud Storage (GCS), ensure files are compressed (GZIP, Snappy) to save space.

Step 2: Uploading the Dataset

Option 1: Upload Directly from Local Storage

- You can upload files directly through the **BigQuery Web UI** or using the **bq command-line tool**.

Option 2: Upload from Google Cloud Storage

- If the dataset is large, upload it first to **Google Cloud Storage (GCS)**.

Step 3: Optimizing Query Performance

Once the data is loaded, follow these best practices for running efficient queries:

1. Partitioning the Table



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

- Partition the table based on **date** or **range** to improve query performance.

2. Clustering the Table

- Cluster on frequently filtered columns.

3. Running Queries Efficiently

- Use **SELECT specific columns** instead of SELECT * to reduce scan costs.
- Use **approximate functions** for large datasets (APPROX_COUNT_DISTINCT() instead of COUNT(DISTINCT column)).
- Use **filters early** in the query (WHERE clause) to reduce data scanning.

Step 4: Automating Data Loading (ETL Pipeline)

For automated and scheduled data loading:

- Use **Cloud Dataflow (Apache Beam)** for complex ETL tasks.
- Use **Cloud Composer (Airflow)** to schedule ETL pipelines.
- Use **BigQuery scheduled queries** for periodic transformations.

Screenshots:

Upload Data from GUI to perform Big Query:

For Csv file format:



SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Add

Source

Q Search for data sources

Popular sources

Local file

Upload a local file

Google Cloud Storage

Google object storage service

Connections to external data sources

Connection from BigQuery to an external data source

PREVIEW

Auto-create external and BigLake tables from Cloud Storage

Automatically scan Cloud Storage buckets to create and update BigLak...

Additional sources

Viewing all 31 results.

Q Search for and star a project

Search for a BigQuery project and add it to the Explorer

Q Star a project by name

Add a BigQuery project to the Explorer by project name

Analytics Hub

Discover and subscribe to public, commercial or privately shared data sets

Google Drive

Google storage service

Salesforce Data Cloud

CLOSE

Create table

Source

Create table from

Upload

Select file *

emp.csv

X BROWSE ?

File format

CSV

Destination

Project *

dataeng09

BROWSE

Data set *

ds1

Table *

emp

Maximum name size is 1,024 UTF-8 bytes. Unicode letters, marks, numbers, connectors, dashes and spaces are allowed.

Table type

Native table

?

Schema

☒ Auto-detect

Schema will be automatically generated.

Partitioning settings

☒ No partitioning

☐ Partition by ingestion time

☐ Partition by field

To partition by field, you must first add fields to the schema above.

CREATE TABLE

CANCEL

Data shown from that CSV:



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

The screenshot shows the Google Cloud BigQuery Studio interface. On the left, the 'Explorer' pane shows the project 'dataeng09' and a table named 'data'. The 'Table Explorer' pane shows the table 'data' with columns: Row, Emp ID, Name Prefix, First Name, Middle Initial, Last Name, Gender, and E Mail. The 'Preview' pane shows the first 50 rows of the table. The 'Job history' pane shows the job history for the table 'data'.

Row	Emp ID	Name Prefix	First Name	Middle Initial	Last Name	Gender	E Mail
27	879753	Mrs.	Pamela	X	Wright	F	pamela.wright@aol.com
28	447813	Mrs.	Ann	Q	Coleman	F	ann.coleman@bmc.com
29	491943	Mrs.	Nancy	P	Howard	F	nancy.howard@gmail.com
30	765850	Mrs.	Linda	R	Moore	F	linda.moore@gmail.com
31	623253	Mrs.	Ruby	Q	Stewart	F	ruby.stewart@gmail.com
32	392491	Mrs.	Theresa	R	Murphy	F	theresa.murphy@gmail.com
33	940761	Ms.	Brenda	S	Robinson	F	brenda.robinson@gmail.com
34	539712	Ms.	Nancy	T	Baker	F	nancy.baker@btp.com
35	214352	Ms.	Theresa	G	Lee	F	theresa.lee@gmail.com
36	979607	Ms.	Carol	J	Edwards	F	carol.edwards@msn.com
37	560455	Ms.	Carolyn	V	Hayes	F	carolyn.hayes@hotmail.co.uk
38	335732	Ms.	Brenda	M	Butler	F	brenda.butler@gmail.com
39	893212	Ms.	Amy	B	Howard	F	amy.howard@aol.com
40	134841	Ms.	Donna	R	Brown	F	donna.brown@aol.com
41	766610	Ms.	Joyce	E	Jenkins	F	joyce.jenkins@aol.com
42	923947	Ms.	Mary	G	Bryant	F	mary.bryant@verizon.net
43	407061	Ms.	Elizabeth	Q	Jackson	F	elizabeth.jackson@aol.com
44	524896	Ms.	Judy	Y	Hernandez	F	judy.hernandez@gmail.com
45	235373	Ms.	Sharon	E	Lopez	F	sharon.lopez@gmail.com
46	515103	Ms.	Anne	A	Perez	F	anne.perez@stcglobal.net
47	687017	Ms.	Frances	P	Watson	F	frances.watson@outlook.com
48	241882	Ms.	Debra	W	Wood	F	debra.wood@hotmail.com
49	732017	Ms.	Cynthia	J	Ramirez	F	cynthia.ramirez@btinternet.com
50	951225	Ms.	Margaret	N	Brooks	F	margaret.brooks@gmail.com

Perform Query on that csv data:

The screenshot shows the Google Cloud BigQuery Studio interface. The 'Query results' pane shows the results of a query: `SELECT * FROM `dataeng09`.`data` LIMIT 1000`. The results are displayed in a table with columns: Row, Emp ID, Name Prefix, First Name, Middle Initial, Last Name, Gender, and E Mail. The table contains 25 rows of data.

Row	Emp ID	Name Prefix	First Name	Middle Initial	Last Name	Gender	E Mail
9	495141	Dr.	Tammy	B	Young	F	tammy.young@comcast.net
10	477616	Hon.	Frances	S	Young	F	frances.young@gmail.com
11	162402	Hon.	Diana	T	Peterson	F	diana.peterson@hotmail.co.uk
12	528509	Hon.	Paula	G	Diaz	F	paula.diaz@gmail.com
13	890290	Hon.	Julia	S	Scott	F	julia.scott@apple.com
14	426038	Hon.	Joan	C	Stewart	F	joan.stewart@yahoo.com
15	329752	Hon.	Lillian	X	Brown	F	lillian.brown@aol.com
16	333476	Hon.	Mary	P	Wilson	F	mary.wilson@stcglobal.net
17	227922	Hon.	Amanda	M	Hughes	F	amanda.hughes@rediffmail.com
18	940922	Hon.	Melissa	X	Butler	F	melissa.butler@hotmail.com
19	478003	Hon.	Cynthia	X	White	F	cynthia.white@gmail.com
20	325319	Hon.	Carolyn	F	Price	F	carolyn.price@yahoo.com
21	969964	Hon.	Janice	S	Parlier	F	janice.parlier@yahoo.com
22	380086	Mrs.	Carol	V	Murphy	F	carol.murphy@gmail.com
23	386158	Mrs.	Melissa	O	King	F	melissa.king@comcast.net
24	441771	Mrs.	Cheryl	O	Scott	F	cheryl.scott@gmail.com
25	388447	Mrs.	Ruby	R	Rivers	F	ruby.rivers@aol.com

For ORC file format:



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar

Roll No: B027

Create table

SAP ID: 60019220125

Branch: CSE(ICB)



Source

Create table from Upload

Select file *
orc1.orc X BROWSE ?

File format
ORC ?

Destination

Project *
dataeng09 BROWSE

Data set *
de1

Table *
orc

Maximum name size is 1,024 UTF-8 bytes. Unicode letters, marks, numbers, connectors, dashes and spaces are allowed.

Table type
Native table ?

Schema

! Source file defines the schema.

Partitioning settings

- ☒ No partitioning
- ☐ Partition by ingestion time
- ☐ Partition by field
To partition by field, you must first add fields to the schema above.

CREATE TABLE CANCEL

The screenshot shows the Google Cloud BigQuery console interface. The left sidebar contains navigation options like Studio, Pipelines and Integration, Governance, Administration, and Migration. The main area displays the 'orc' table schema in the 'dataeng09' project. The schema table lists fields with their names, types, modes, keys, collations, default values, policy tags, and descriptions.

Field name	Type	Mode	Key	Collation	Default value	Policy tags	Description
boolean1	BOOLEAN	NULLABLE	-	-	-	-	-
byte1	INTEGER	NULLABLE	-	-	-	-	-
short1	INTEGER	NULLABLE	-	-	-	-	-
int1	INTEGER	NULLABLE	-	-	-	-	-
long1	INTEGER	NULLABLE	-	-	-	-	-
float1	FLOAT	NULLABLE	-	-	-	-	-
double1	FLOAT	NULLABLE	-	-	-	-	-
bytes1	BYTES	NULLABLE	-	-	-	-	-
string1	STRING	NULLABLE	-	-	-	-	-
middle	RECORD	NULLABLE	-	-	-	-	-
list	RECORD	REPEATED	-	-	-	-	-
map	RECORD	REPEATED	-	-	-	-	-
ts	TIMESTAMP	NULLABLE	-	-	-	-	-
decimal1	NUMERIC	NULLABLE	-	-	-	-	-



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Google Cloud BigQuery Explorer view showing a table named 'orc' in the 'data' dataset. The table has 12 rows and 13 columns. The columns are: Row, boolean, byte1, short1, int1, long1, float1, double1, LINEAGE, DATA PROFILE, DATA QUALITY, string1, mode1, set1, middle1, string1. The table is currently empty.

Row	boolean	byte1	short1	int1	long1	float1	double1	LINEAGE	DATA PROFILE	DATA QUALITY	string1	mode1	set1	middle1	string1
1	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
2	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
3	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
4	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
5	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
6	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
7	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
8	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
9	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
10	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
11	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye
12	false			1024	65536	922337203...	1.0	-15.0	AAECAwQ=		NI			1	bye

Perform Query on ORC file:

Google Cloud BigQuery Explorer view showing a query executed on the 'orc' table. The query is 'SELECT * FROM `dataeng09`.`orc` LIMIT 1000'. The results show 8 rows of data.

Row	boolean	byte1	short1	int1	long1	float1	double1	LINEAGE	DATA PROFILE	DATA QUALITY	string1	mode1	set1	middle1	string1
1	false			1024	65536	9223372036854...	1.0	-15.0	AAECAwQ=		NI			1	bye
2	false			1024	65536	9223372036854...	1.0	-15.0	AAECAwQ=		NI			1	bye
3	false			1024	65536	9223372036854...	1.0	-15.0	AAECAwQ=		NI			1	bye
4	false			1024	65536	9223372036854...	1.0	-15.0	AAECAwQ=		NI			1	bye
5	false			1024	65536	9223372036854...	1.0	-15.0	AAECAwQ=		NI			1	bye
6	false			1024	65536	9223372036854...	1.0	-15.0	AAECAwQ=		NI			1	bye
7	false			1024	65536	9223372036854...	1.0	-15.0	AAECAwQ=		NI			1	bye
8	false			1024	65536	9223372036854...	1.0	-15.0	AAECAwQ=		NI			1	bye

Bucket Creation and Upload data in it:



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Google Cloud

My First Project

bucket

Cloud Storage

Create a Bucket

Overview

Buckets

Monitoring

Settings

Get started

Name: bucket_tushar_jcb

Choose where to store your data

Location: asia-south1 (Mumbai)

Location type: Region

Choose a storage class for your data

Default storage class: Standard

Choose how to control access to objects

Public access prevention: Off

Access control: Uniform

Choose how to protect object data

Your data is always protected with Cloud Storage but you can also choose from these additional data protection options to add extra layers of security.

Data protection

Soft-delete policy (for data recovery)

When enabled, this bucket and its objects will be kept for a specified period after they're deleted and can be restored during this time. [Learn more](#)

Use default retention duration

All buckets have a seven-day soft delete duration by default, unless this default has been customised by your organisation administrator.

Set custom retention duration

Specify how long this bucket and its objects should be retained after they're deleted. Setting a '0' duration disables soft delete, meaning that any deleted objects will be deleted permanently.

Object versioning (for version control)

For restoring deleted or overwritten objects. To minimise the cost of storing versions, we recommend limiting the number of non-current versions per object and scheduling them to expire after a number of days. [Learn more](#)

Retention (for compliance)

For preventing the deletion or modification of the bucket's objects for a specified period of time.

DATA ENCRYPTION

Good to know

Location pricing

Storage rates vary depending on the storage class of your data and location of your bucket. [Pricing details](#)

Current configuration: Region / Standard

Item	Cost
asia-south1 (Mumbai)	\$0.023 per GB/month

ESTIMATE YOUR MONTHLY COST

CREATE

CANCEL

bucket_tushar_jcb - Bucket details

Inbox (889) - tpancha2035@gmail.com

console.cloud.google.com/storage/browser/bucket_tushar_jcb?tab=objects?project=festive-quanta-451305-g4&prefix=&forceOnObjectsSortingFiltering=false

Google Cloud

My First Project

bucket

Search

Cloud Storage

Bucket details

GO TO PATH

REFRESH

LEARN

Overview

Buckets

Monitoring

Settings

bucket_tushar_jcb

Location: asia-south1 (Mumbai)

Storage class: Standard

Public access: Not public

Protection: Soft delete

OBJECTS

CONFIGURATION

PERMISSIONS

PROTECTION

LIFECYCLE

OBSERVABILITY

INVENTORY REPORTS

OPERATIONS

Folder browser

bucket_tushar_jcb

CREATE FOLDER

UPLOAD

TRANSFER DATA

OTHER SERVICES

Filter by name prefix only

Filter objects and folders

Show Live objects only

Name	Size	Type	Created	Storage class	Last modified	Public access	Version history	Encryption	Object retention retain until time	Retention expiry time
No rows to display										

Created bucket bucket_tushar_jcb

Your bucket is ready. Just add data.



SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Add



Source

Search for data sources

Popular sources



Local file

Upload a local file



Google Cloud Storage

Google object storage service



Connections to external data
sources

Connection from BigQuery to an
external data source



PREVIEW

Auto-create external and BigLake
tables from Cloud Storage

Automatically scan Cloud Storage
buckets to create and update BigLake...

Create table

Source

Create table from
Google Cloud Storage

Select file from GCS bucket or [use a URI pattern](#)

File format
Parquet

☐ Source data partitioning

Destination

Project *
festive-quanta-451305-g4

Data set *

Table *

Maximum name size is 1,024 UTF-8 bytes. Unicode letters, marks, numbers, connectors, dashes and spaces are allowed.

Table type
Native table

Schema

Source file defines the schema.

Partitioning settings

☒ No partitioning
☐ Partition by ingestion time
☐ Partition by field
To partition by field, you must first add fields to the schema above.

CREATE TABLE CANCEL

Choose a file

bucket_tushar_icb



mtcars.parquet

Filename
mtcars.parquet

SELECT CANCEL



SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Create table

Source

Create table from
Google Cloud Storage

Select file from GCS bucket or use a URI pattern [?](#) *
☒ bucket_tushar_icb/mtcars.parquet

BROWSE ?

File format
Parquet

☐ Source data partitioning

Destination

Project *
festive-quanta-451305-g4

BROWSE

Data set *
bucket_1

Table *
slide_1
Maximum name size is 1,024 UTF-8 bytes. Unicode letters, marks, numbers, connectors, dashes and spaces are allowed.

Table type
Native table

Schema

Source file defines the schema.

Partitioning settings

☒ No partitioning
☐ Partition by ingestion time
☐ Partition by field
To partition by field, you must first add fields to the schema above.

CREATE TABLE

CANCEL

bucket_tushar_icb

Location
asia-south1 (Mumbai)

Storage class
Standard

Public access
Not public

Protection
Soft delete

OBJECTS

CONFIGURATION

PERMISSIONS

PROTECTION

LIFECYCLE

OBSERVABILITY

INVENTORY REPORTS

OPERATIONS

Folder browser

bucket_tushar_icb

CREATE FOLDER

UPLOAD

TRANSFER DATA

OTHER SERVICES

Filter by name prefix only

Filter

Filter objects and folders

Show Live objects only

☐	Name	Size	Type	Created	Storage class	Last modified	Public access	Version history	
☐	mtcars.parquet	4.5 KB	application/octet-stream	18 Feb 2025, 11:34:56	Standard	18 Feb 2025, 11:34:56	Not public	—	⬇ ⋮



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

The screenshot shows the Google Cloud BigQuery Explorer interface. The left sidebar displays the project hierarchy: 'festive-quanta-451305-g4' > 'bucket_1' > 'slide_1'. The main panel shows the 'slide_1' table with columns: model, mpg, cyl, disp, hp, drat, wt, qsec, vs, am, gear, carb. The table contains 27 rows of data for various car models like Merc 240D, Merc 230, Toyota Corona, etc. The bottom status bar shows '29°C Smoke' and system time '11:30 18-02-2025'.

Perform Query on that Bucket Data:

The screenshot shows the Google Cloud BigQuery Explorer interface with a query executed. The query is: `SELECT * FROM 'festive-quanta-451305-g4.bucket_1.slide_1' LIMIT 1000`. The 'Query results' section displays the first 13 rows of the table, matching the data in the first screenshot. The bottom status bar shows '29°C Smoke' and system time '11:30 18-02-2025'.

Create Pipeline on Google Cloud:



SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

[←](#) Create repository

Repositories contain a single Dataform project that can be connected to your Git provider. Within a repository, you can create workspaces for development and execute your SQL workflows against BigQuery.

Repository ID *

tushar_001

Region *

asia-south1 (Mumbai)

Service account

Default Dataform service account

☐ Use the default KMS key for encryption

Encryption
Google-managed encryption key will be ignored if the default customer-managed encryption key is set in this project location.
☒ Google-managed encryption key
Keys owned by Google
☐ Cloud KMS key
Keys owned by customers

New repositories start empty and can be connected to a git provider after being created.

Creating...

Google Cloud My First Project bucke

🔍

← Create repository

📁

•

⚙️

➡️

🕒

📋

🔍

🔒

•

•

☰

🔧

✅ Repository created successfully.

🔔 Dataform will execute workflows as the service account service-866073429415@gcp-sa-dataform.iam.gserviceaccount.com.

⚠️ To execute queries in Dataform, you need the following roles:
for service-866073429415@gcp-sa-dataform.iam.gserviceaccount.com:

- roles/bigquery.jobUser on festive-quanta-451305-g4

Grant all Learn more

Go to repositories



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

The screenshot shows the Google Cloud Dataform console. The left sidebar contains navigation options: Studio, Pipelines & Integration, Data transfers, Pipelines (Dataform), Scheduled queries, Scheduling, Governance, Analytics Hub, Policy tags, Administration, Migration, Assessment, SQL translation, Partner Center, Settings, and Release Notes. The main panel displays the 'Filter repositories' table with columns: Name, Location, Source, Authentication status, and Action. A single repository 'tushar_001' is listed with location 'asia-south1'. The right sidebar shows 'Recommended for you' with links to get started, create and execute a workflow, schedule production executions, create development environments, and learn more.

The screenshot shows the Google Cloud Dataform console for the 'tushar_001' repository. The left sidebar is the same as the previous screenshot. The main panel displays the 'Development Workspaces' section with a table containing one workspace: 'tushar_devarsh_01'. The right sidebar shows 'Recommended for you' with links to create a workspace, create a release configuration, create a workflow configuration, monitor execution logs, and configure workspace compilation overrides.

The screenshot shows the Google Cloud Dataform console for the 'tushar_devarsh_01' repository, specifically the 'Compiled graph' view. The main panel displays a graph with two nodes: 'first_view' and 'second_view', both with the view name 'tushar_devarsh_01'. The right sidebar shows the 'Query' tab with a sample SQL query: 'SELECT 1 as test'.

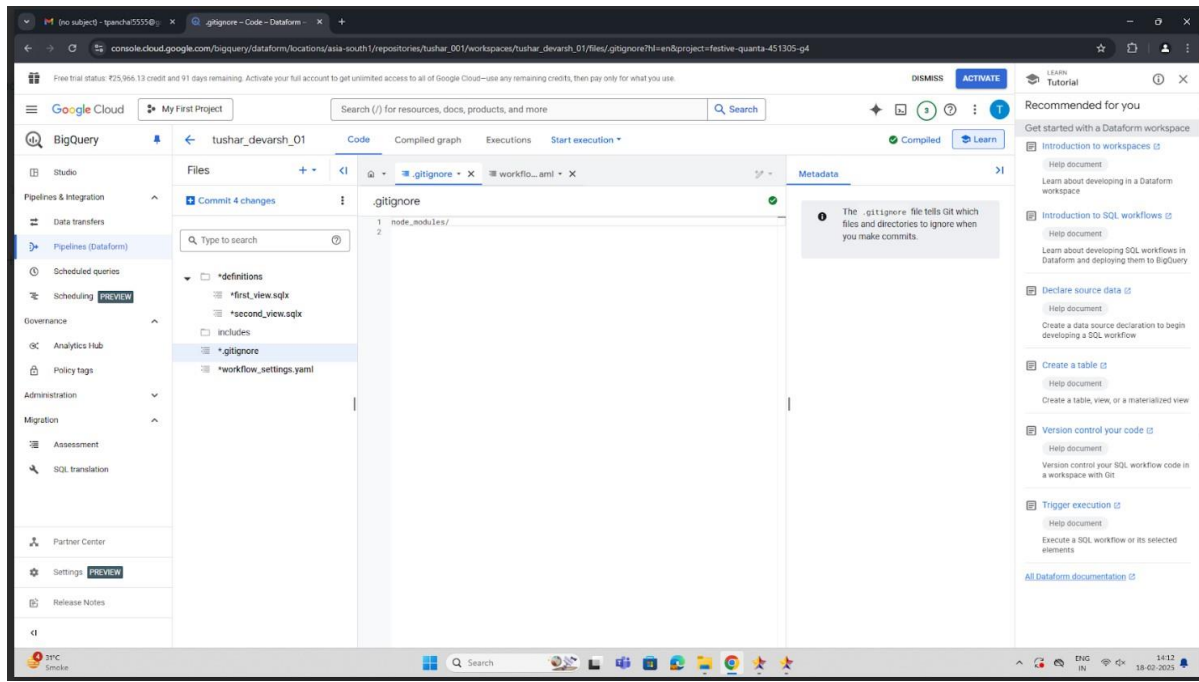


**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)



Schedule Query:



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Export as scheduled query

✔ Settings valid.

Details and schedule

Name for scheduled query *

count_all_q

Schedule options

Repeat frequency *

Minutes

Repeat frequency *

5

minutes



☒ Start now ☐ Start at set time

Start date and run time

18/02/2025, 11:45

IST



☐ End never ☒ Schedule end time

End date *

19/02/2025, 00:00

IST



Scheduled to run every 5 minutes starting now and ending on 19 February 2025 at 00:00:00 IST.

Destination for query results

☒ Set a destination table for query results

Dataset *

☒ festive-quanta-451305-g4.bucket_1

Table Id *

scheduled_time

Destination table partitioning field

Destination table write preference

☒ Append to table

☐ Overwrite table

SAVE

CANCEL



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

The screenshot shows the Google Cloud BigQuery console interface. The query editor at the top contains the following SQL code:

```
1 SELECT COUNT(*)  
2 FROM `festive-quanta-451305-g4.bucket_1.slide_1`;  
3
```

The 'Query results' section shows a single row with the value 32. The 'Job history' section lists several jobs, including the current one 'count_all_q' which is marked as 'Succeeded'.

Scheduled queries							
Filter Filter scheduled query configs							
☐	☒	Display name	Source	Schedule (UTC)	Region	Destination data set	Next scheduled UTC+5:30
☐	☒	count_all_q	Scheduled Query	every 5 minutes	asia-south1	bucket_1	18 February 2025 at 12:00:00 UTC+5:30

count_all_q				
Succeeded e (UTC) Target date for next run				
every 5 minutes 18 February 2025 at 12:00:00 UTC+5:30				
Filter Filter scheduled query runs				
☒	Run date	Schedule time	UTC+5:30	Summary
☒	18 February 2025	18 February 2025 at 11:55:00 UTC+5:30		The scheduled query has completed successfully.

Conclusion

Thus we successfully implemented loading a large dataset into a data warehouse and ran queries on it and implemented SQL-based transformations using data build tool to build a reliable data transformation online.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 4

Aim: To build a data pipeline that applies multiple transformation to a dataset creating a repeatable process using Apache Airflow.

Theory:

Apache Airflow is an open-source workflow automation tool designed to orchestrate, schedule, and monitor workflows (DAGs - Directed Acyclic Graphs). It allows users to define workflows as Python code, making them dynamic and scalable. Airflow ensures efficient execution of complex data workflows by managing dependencies, retries, and parallel executions.

Key Features of Apache Airflow

- Scalability: Can handle large-scale workflows efficiently.
- Extensibility: Integrates with cloud services like AWS, GCP, and Azure.
- UI & Monitoring: Provides a web-based interface for managing DAGs.
- Task Dependencies: Automatically resolves task execution order.
- Retry Mechanism: Retries failed tasks to ensure pipeline reliability.
- Parallel Execution: Supports multiple task execution using Celery Executor.

Implementation of a Pipeline in Apache Airflow

A pipeline in Apache Airflow is defined using DAGs. A DAG is a collection of tasks, executed in a specified order based on dependencies.

Steps to Implement a Pipeline:

1. Install Apache Airflow
 - o Install Airflow using pip and set up a virtual environment.
 - Initialize the database and start the webserver & scheduler.
2. Define a DAG (Workflow Pipeline)
 - o Create a DAG file in `~/airflow/dags/`.
 - o Define tasks using operators (PythonOperator, BashOperator, etc.).
 - Set task dependencies (`task_1 >> task_2` for sequential execution).
3. Start Airflow Services
 - o Run `airflow webserver --port 8080` to launch the UI.



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

- Run airflow scheduler to start executing workflows.
4. Trigger and Monitor the DAG o Access <http://localhost:8080>, enable the DAG, and trigger execution.
- Monitor tasks using the UI (Graph View, Tree View, Log

Screenshots:

1) Apache installation:

```
student@MUM0922CPU0070:~$ python3 -m venv airflow_venv
source airflow_venv/bin/activate
The virtual environment was not created successfully because ensurepip is not
available. On Debian/Ubuntu systems, you need to install the python3-venv
package using the following command.

apt install python3.8-venv

You may need to use sudo with that command. After installing the python3-venv
package, recreate your virtual environment.

Failing command: ['/home/student/airflow_venv/bin/python3', '-Im', 'ensurepip', '--upgrade', '--default-pip']

student@MUM0922CPU0070:~$ source airflow_venv/bin/activate
-bash: airflow_venv/bin/activate: No such file or directory
student@MUM0922CPU0070:~$ ^V^C
student@MUM0922CPU0070:~$ sudo apt update && sudo apt upgrade -y
Get:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Hit:2 http://archive.ubuntu.com/ubuntu focal InRelease
Get:3 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:4 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages [3421 kB]
```

```
(airflow_venv) student@MUM0922CPU0070:~$ airflow db init
/home/student/airflow_venv/lib/python3.8/site-packages/airflow/cli/commands/db_command.py:43 DeprecationWarning: 'db init' is deprecated. Use 'db
to create the default connections
DB: sqlite:///home/student/airflow/airflow.db
[2025-03-04T11:24:37.090+0530] {migration.py:213} INFO - Context impl SQLiteImpl.
[2025-03-04T11:24:37.105+0530] {migration.py:216} INFO - Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running stamp_revision -> 405de8318b3a
WARNI [airflow.models.crypto] empty cryptography key - values will not be stored encrypted.
Initialization done
(airflow_venv) student@MUM0922CPU0070:~$ airflow users create \
> --username admin \
> --password admin123 \
> --firstname Admin \
> --lastname User \
> --role Admin \
> --email admin@example.com
/home/student/airflow_venv/lib/python3.8/site-packages/flask_limiter/extension.py:336 UserWarning: Using the in-memory storage for tracking rate l
ee: https://flask-limiter.readthedocs.io#configuring-a-storage-backend for documentation about configuring the storage backend.
[2025-03-04T11:24:37.090+0530] {manager.py:239} INFO - Inserted Role: %s
[2025-03-04T11:24:37.105+0530] {manager.py:239} INFO - Inserted Role: %s
[2025-03-04T11:24:37.107+0530] {manager.py:670} WARNING - No user yet created, use flask fab command to do it.
[2025-03-04T11:24:37.200+0530] {manager.py:499} INFO - Created Permission View: %s
[2025-03-04T11:24:37.214+0530] {manager.py:555} INFO - Added Permission %s to role %s
[2025-03-04T11:24:37.241+0530] {manager.py:499} INFO - Created Permission View: %s
```

```
airflow users create \
```



Name: Gargi Sukhatankar

Roll No: B027

--username admin \

--password admin123 \

--firstname Admin \

--lastname User \

--role Admin \

--email admin@example.com

SAP ID: 60019220125

Branch: CSE(ICB)

1. In the first terminal, start the Airflow web server:

```
airflow webserver --port 8080
```

2. In the second terminal, start the Airflow scheduler:

```
airflow scheduler
```

Apache airflow webserver

Pipeline implementation:

```
cd ~/airflow/dags
```

```
nano my_pipeline.py
```

```
from airflow import DAG
```

```
from airflow.operators.dummy import DummyOperator
```

```
from airflow.operators.python import PythonOperator
```

```
from datetime import datetime, timedelta
```

```
# Function for Python Task
```

```
def my_task():
```

```
print("Executing my custom task in Airflow")
```

```
# Define default arguments
```

```
default_args = {
```

```
"owner": "airflow",
```



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

The screenshot shows the Apache Airflow web interface. The top navigation bar includes links for DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The main header displays the DAG name 'my_pipeline_Vipul' and its schedule '1 day, 0:00:00'. Below the header, there are tabs for Grid, Graph, Calendar, Task Duration, Task Times, Landing Times, Gantt, Details, Code, and Audit Log. The 'Details' tab is selected, showing a summary of the DAG. The summary includes the following information:

DAG Summary	
Total Tasks	3
Empty Operators	2
PythonOperator	1

Below the summary, there are details about the DAG:

DAG Details	
Dag id	my_pipeline_Vipul
Description	A simple Airflow pipeline
Fileloc	/home/student/airflow/dags/my_pipeline_Vipul.py
Has import errors	false
Has task concurrency limits	false
Is active	true
Is paused	true
Is subdag	false

The screenshot shows a list of DAGs in the Apache Airflow web interface. The list includes the following DAGs:

DAG Name	Trigger	Frequency	Last Run
example_trigger_target_dag	airflow	None	
example_weekday_branch_operator	airflow	@daily	2025-03-03 00:00:00+00:00
example_xcom	airflow	@once	2021-01-01 00:00:00+00:00
example_xcom_args	airflow	None	
example_xcom_args_with_operators	airflow	None	
latest_only	airflow	4:00:00	2025-03-04 05:02:54.078582+00:00
latest_only_with_trigger	airflow	4:00:00	2025-03-04 05:02:52.902129+00:00
my_pipeline	airflow	1 day, 0:00:00	2025-03-04 00:00:00+00:00
my_pipeline_vipul	airflow	1 day, 0:00:00	2025-03-04 00:00:00+00:00
tutorial	airflow	1 day, 0:00:00	2025-03-03 09:02:28.429881+00:00
tutorial_dag	airflow	None	
tutorial_taskflow_api	airflow	None	

At the bottom of the page, it says 'Showing 1-12 of 52 DAGs'.

"depends_on_past": False,
"start_date": datetime(2025, 3, 4),



Name: Gargi Sukhatankar

Roll No: B027

"retries": 1,

"retry_delay": timedelta(minutes=5),

}

Define the DAG

with DAG(

"my_pipeline",

default_args=default_args,

description="A simple Airflow pipeline",

schedule_interval=timedelta(days=1), # Runs daily

catchup=False,

) as dag:

start = DummyOperator(task_id="start")

task_1 = PythonOperator(

task_id="my_python_task",

python_callable=my_task,

)

end = DummyOperator(task_id="end")

Define Task Order

start >> task_1 >> end

Conclusion:

Thus we successfully built a data pipeline that applies multiple transformation to a dataset creating a repeatable process using Apache Airflow.

SAP ID: 60019220125

Branch: CSE(ICB)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 5

Aim: Implement least privilege principle to minimize the number of permissions granted to users by using database tool.

Theory:

Implementing the Principle of Least Privilege (PoLP) in a Database Using MySQL Workbench

Understanding the Principle of Least Privilege (PoLP)

The Principle of Least Privilege (PoLP) is a fundamental security approach that restricts users, applications, and systems to only the essential permissions required for their specific functions. In database management, implementing PoLP reduces exposure to security threats, prevents unauthorized modifications, and enhances data integrity.

By leveraging MySQL Workbench, administrators can systematically assign and control access rights, ensuring that users operate within predefined boundaries to maintain a secure and efficient database environment.

Stage 1: Identify User Roles and Permissions

1. Analyze job functions and determine the minimum permissions each role needs.
2. Create a list of database operations each role requires (e.g., READ, WRITE, DELETE).

Stage 2: Create Users and Assign Minimal Permissions

Use a database management tool such as **MySQL Workbench**, **SQL Server Management Studio (SSMS)**, or **pgAdmin** to create and configure users with only necessary privileges.

A. Example 1: MySQL

-- Create a new user

```
CREATE USER 'app_user'@'localhost' IDENTIFIED BY 'StrongPassword';
```

-- Grant only SELECT permission on a specific database

```
GRANT SELECT ON company_db.* TO 'app_user'@'localhost';
```

-- Apply changes

```
FLUSH PRIVILEGES;
```



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Stage 3: Implement Role-Based Access Control (RBAC)

Instead of assigning permissions directly to users, assign them to roles.

Example: Assigning Roles in MySQL

```
-- Create a role with minimal privileges
CREATE ROLE read_role;

-- Grant SELECT permission to the role
GRANT SELECT ON company_db.* TO read_role;

-- Assign the role to a user
GRANT read_role TO 'app_user'@'localhost';

-- Set default role (optional)
SET DEFAULT ROLE read_role FOR 'app_user'@'localhost';
```

Stage 4: Regularly Audit and Revoke Unnecessary Permissions

Check User Permissions

□ MySQL:

```
SHOW GRANTS FOR 'app_user'@'localhost';
```

Revoke Excessive Permissions

```
REVOKE INSERT, UPDATE, DELETE ON company_db.* FROM 'app_user'@'localhost';
```

Stage 5: Monitor and Enforce Access Controls

- Use **logging and auditing tools** to track access attempts.
- Enable **Database Activity Monitoring (DAM)**. □
Set up **alerts** for privilege escalations.

Output:



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

34 • `select *from students;`

34 • `select *from students;`

Result Grid		Filter Rows:	Edit:	
	roll_no	name	address	blood_group
▶	1	Aarav Mehta	Mumbai, India	A +
	2	Sneha Sharma	Delhi, India	B +
	3	Rohan Patel	Ahmedabad, India	O +
	4	Meera Iyer	Chennai, India	AB +
	5	Vikram Singh	Jaipur, India	A -
	6	Pooja Nair	Kolkata, India	B -
	7	Amit Das	Bangalore, India	O -
	8	Ritu Verma	Pune, India	AB -
	9	Kunal Thakur	Hyderabad, India	A +
	10	Neha Joshi	Lucknow, India	B +
	11	Rahul Kapoor	Indore, India	O +
	12	Simran Malhotra	Chandigarh, India	AB +
	13	Arjun Desai	Surat, India	A -
	14	Priya Saxena	Bhopal, India	B -
	15	Manoj Gupta	Nagpur, India	O -
	16	Swati Kulkarni	Patna, India	AB -
	17	Deepak Reddy	Vishakhapatnam, ...	A +
	18	Nisha Yadav	Coimbatore, India	B +
	19	Rakesh Sen	Guwahati, India	O +
	20	Sonia Dutta	Raipur, India	AB +

36 • `CREATE USER 'root'@'3306' IDENTIFIED BY 'StrongPassword123';`

8 11:14:03 CREATE USER 'root'@'3306' IDENTIFIED BY 'StrongPassword123'

0 row(s) affected

38 • `GRANT SELECT ON student_db.* TO 'root'@'3306';`

9 11:14:40 GRANT SELECT ON student_db.* TO 'root'@'3306'

0 row(s) affected

40 • `SHOW GRANTS FOR 'root'@'3306';`

Result Grid	Filter Rows:	Export:
Grants for root@3306		
▶	GRANT USAGE ON *.* TO 'root'@'3306'	
	GRANT SELECT ON `student_db`.* TO 'root'...	



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

```
SELECT * FROM student_db.students;

SELECT * FROM student_db.students;

47 • INSERT INTO students VALUES (21, 'Raj Malhotra', 'Bangalore, India', 'O+');

47 • INSERT INTO students VALUES (21, 'Raj Malhotra', 'Bangalore, India', 'O+');

-- User 2: data_entry (Insert & Select Privileges);
CREATE USER 'data_entry'@'localhost' IDENTIFIED BY 'EntryPass456';

16 11:34:05 CREATE USER 'data_entry'@'localhost' IDENTIFIED BY 'EntryPass456' 0 row(s) affected

GRANT SELECT, INSERT ON student_db.* TO 'data_entry'@'localhost';

• FLUSH PRIVILEGES;

18 11:39:06 FLUSH PRIVILEGES

-- User 3: admin_user (Full Control)
CREATE USER 'admin_user'@'3306' IDENTIFIED BY 'AdminPass789';

19 11:43:30 CREATE USER 'admin_user'@'3306' IDENTIFIED BY 'AdminPass789' 0 row(s) affected

GRANT ALL PRIVILEGES ON student_db.* TO 'admin_user'@'3306';

22 11:45:52 GRANT ALL PRIVILEGES ON student_db.* TO 'admin_user'@'3306'

-- Verify Permissions for Each User

• SHOW GRANTS FOR 'root'@'3306';
• SHOW GRANTS FOR 'data_entry'@'3306';
• SHOW GRANTS FOR 'admin_user'@'3306';
```

Result Grid	Filter Rows:	Export:
Grants for root@3306		
GRANT USAGE ON *,* TO 'root'@'3306'		
GRANT SELECT ON 'student_db',* TO 'root'...		

Result Grid	Filter Rows:	Export:
Grants for data_entry@3306		
GRANT USAGE ON *,* TO 'data_entry'@'3306'		

Result Grid	Filter Rows:	Export:
Grants for admin_user@3306		
GRANT USAGE ON *,* TO 'admin_user'@'3306'		
GRANT ALL PRIVILEGES ON 'student_db',* TO...		



SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

```
-- Role-Based Access Control (RBAC)

-- Create Roles

CREATE ROLE read_role;
CREATE ROLE entry_role;
CREATE ROLE admin_role;

40 11:53:28 CREATE ROLE read_role
41 11:53:31 CREATE ROLE entry_role
42 11:53:35 CREATE ROLE admin_role
-- Assign Privileges to Roles

GRANT SELECT ON student_db.* TO read_role;
GRANT SELECT, INSERT ON student_db.* TO entry_role;
GRANT ALL PRIVILEGES ON student_db.* TO admin_role;

43 11:58:18 GRANT SELECT ON student_db.* TO read_role
44 11:58:20 GRANT SELECT, INSERT ON student_db.* TO entry_role
45 11:58:22 GRANT ALL PRIVILEGES ON student_db.* TO admin_role
-- Assign Roles to Users

GRANT read_role TO 'root'@'3306';
GRANT entry_role TO 'data_entry'@'3306';
GRANT admin_role TO 'admin_user'@'3306';

46 12:00:31 GRANT read_role TO 'root'@'3306'
47 12:00:33 GRANT entry_role TO 'data_entry'@'3306'
48 12:00:35 GRANT admin_role TO 'admin_user'@'3306'

-- Set Default Roles

SET DEFAULT ROLE read_role TO 'root'@'3306';
SET DEFAULT ROLE entry_role TO 'data_entry'@'3306';
SET DEFAULT ROLE admin_role TO 'admin_user'@'3306';

50 12:03:40 SET DEFAULT ROLE read_role TO 'root'@'3306' 0 row(s) affected
51 12:03:43 SET DEFAULT ROLE entry_role TO 'data_entry'@'3306' 0 row(s) affected
52 12:03:45 SET DEFAULT ROLE admin_role TO 'admin_user'@'3306' 0 row(s) affected

-- Auditing and Security Checks

SHOW GRANTS FOR 'root'@'3306';
SHOW GRANTS FOR 'data_entry'@'3306';
SHOW GRANTS FOR 'admin_user'@'3306';
```



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Result Grid	Filter Rows:	Export:
Grants for root@3306		
▶	GRANT USAGE ON *.* TO 'root'@'3306'	
	GRANT SELECT ON 'student_db',* TO 'root'...	
	GRANT 'read_role'@'%' TO 'root'@'3306'	

Result Grid	Filter Rows:	Export:
Grants for data_entry@3306		
▶	GRANT USAGE ON *.* TO 'data_entry'@'3306'	
	GRANT 'entry_role'@'%' TO 'data_entry'@...	

Result Grid	Filter Rows:	Export:
Grants for admin_user@3306		
▶	GRANT USAGE ON *.* TO 'admin_user'@'3306'	
	GRANT ALL PRIVILEGES ON 'student_db',* TO...	
	GRANT 'admin_role'@'%' TO 'admin_user'...	

-- To check who has access to what:

```
SELECT user, host FROM mysql.user;
```

Result Grid	Filter Rows:	Export:	Wrap
	user	host	
▶	admin_role	%	
	entry_role	%	
	read_role	%	
	admin_user	3306	
	data_entry	3306	
	root	3306	
	data_entry	localhost	
	mysql.infoschema	localhost	
	mysql.session	localhost	
	mysql.sys	localhost	
	root	localhost	

user 14 x
89 • SHOW VARIABLES LIKE 'general_log';

Result Grid	Filter Rows:	Export:	Wr
	Variable_name	Value	
▶	general_log	ON	



SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

```
-- monitor login attempts |
```

```
SELECT * FROM mysql.general_log WHERE command_type = 'Connect';
```

Result Grid						
Filter Rows:		Export:		Wrap Cell Content:		
event_time	user_host	thread_id	server_id	command_type	argument	
2025-03-11 12:16:30.554331	[root] @ localhost [::1]	19	1	Connect	BLOB	
2025-03-11 12:16:30.557742	root[root] @ localhost [::1]	19	1	Connect	BLOB	
2025-03-11 12:16:30.736487	[root] @ localhost [::1]	20	1	Connect	BLOB	
2025-03-11 12:16:30.737267	root[root] @ localhost [::1]	20	1	Connect	BLOB	
2025-03-11 12:16:41.231285	[root] @ localhost [::1]	21	1	Connect	BLOB	
2025-03-11 12:16:41.232722	root[root] @ localhost [::1]	21	1	Connect	BLOB	
2025-03-11 12:16:46.246490	[root] @ localhost [::1]	22	1	Connect	BLOB	
2025-03-11 12:16:46.316651	[root] @ localhost [::1]	23	1	Connect	BLOB	

```
-- Revoke Unnecessary Permissions
```

```
REVOKE INSERT, UPDATE, DELETE ON student_db.* FROM 'root'@'3306';
```

9 12:28:34 REVOKE INSERT, UPDATE, DELETE ON student_db.* FROM 'root'@'3306'

0 row(s) affected



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

```
23 • SELECT *
24 FROM information_schema.schema_privileges
25 WHERE grantee = "'admin_user'@'3306'"
26 AND table_schema = 'student_db';
```

Result Grid					
		Filter Rows:		Export:	Wrap Cell Content: IA
	GRANTEE	TABLE_CATALOG	TABLE_SCHEMA	PRIVILEGE_TYPE	IS_GRANTABLE
▶	'admin_user'@'3306'	def	student_db	SELECT	NO
	'admin_user'@'3306'	def	student_db	CREATE	NO
	'admin_user'@'3306'	def	student_db	DROP	NO
	'admin_user'@'3306'	def	student_db	REFERENCES	NO
	'admin_user'@'3306'	def	student_db	INDEX	NO
	'admin_user'@'3306'	def	student_db	ALTER	NO
	'admin_user'@'3306'	def	student_db	CREATE TEMPORARY TABLES	NO
	'admin_user'@'3306'	def	student_db	LOCK TABLES	NO
	'admin_user'@'3306'	def	student_db	EXECUTE	NO
	'admin_user'@'3306'	def	student_db	CREATE VIEW	NO
	'admin_user'@'3306'	def	student_db	SHOW VIEW	NO
	'admin_user'@'3306'	def	student_db	CREATE ROUTINE	NO
	'admin_user'@'3306'	def	student_db	ALTER ROUTINE	NO

```
23 • SELECT *
24 FROM information_schema.schema_privileges
25 WHERE grantee = "'root'@'3306'"
26 AND table_schema = 'student_db';
```

Result Grid					
		Filter Rows:		Export:	Wrap Cell Content: IA
	GRANTEE	TABLE_CATALOG	TABLE_SCHEMA	PRIVILEGE_TYPE	IS_GRANTABLE
▶	'root'@'3306'	def	student_db	SELECT	NO

Conclusion:

Thus we implemented PoLP (Principle of least Privilege) in MySQL Workbench. Aim: Implement least privilege principle to minimize the number of permissions granted to users by using database tool. give conclusion for this above aim.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 6

Aim: Data Anonymization Using Masking and Redaction Techniques in Data Engineering.

Theory:

- Implementing data anonymization using masking and redaction techniques is crucial for protecting sensitive information in data engineering. Here's a step-by-step guide to help you get started:
- Understand the Techniques
 - Data Masking: This involves replacing sensitive data with fictional but realistic data. For example, replacing a real credit card number with a fake one that has the same format.
 - Data Redaction: This involves permanently removing or obscuring sensitive information. For example, blacking out parts of a document.
- Identify Sensitive Data
- Determine which data needs to be anonymized. This typically includes personally identifiable information (PII), financial data, and health records.
- Choose the Right Tools
- Use tools and libraries that support data masking and redaction. Some popular options include:
 - Apache NiFi: For data flow automation and transformation.
 - Talend: For data integration and anonymization.
 - PII Tools: For de-identification, masking, and redaction (<https://pii-tools.com/pii-de-identification-vs-masking-vs-redaction/>).
- Python API

Implement Data Masking

- Static Data Masking: Apply masking to data at rest. This is useful for non-production environments like testing and development.
- Dynamic Data Masking: Apply masking on-the-fly to data in use. This is useful for production environments where data needs to be accessed but not exposed.
- Implement Data Redaction
- Manual Redaction: Manually redact sensitive information in documents and files. This is often used for legal and compliance purposes.



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

- Automated Redaction: Use software to automatically detect and redact sensitive information in large datasets and documents (<https://bigid.com/blog/data-redaction/>).
-
- Test and Validate
- Ensure that the anonymized data retains its usability for analysis and processing.
- Validate that the masking and redaction processes effectively protect sensitive information.
- Monitor and Maintain
- Regularly review and update your anonymization processes to adapt to new types of sensitive data and regulatory requirements.
- By following above steps, you can effectively implement data anonymization using masking and redaction techniques in your data engineering projects. If you have any specific questions or need further details, feel free to ask! escalations.

OUTPUT:

Step 1: Install Required Libraries

```
[42] import pandas as pd
import re
from faker import Faker
```

Step 2: Load the CSV

```
[41] import pandas as pd

df = pd.read_csv("/content/employee_info.csv")
df.head()
```

	Name	Email	Phone
0	Tiya	tiya@gmail.com	9876543210
1	Eevee	eevee@example.com	9123456789
2	Ash	ash@poke.com	9988776655

Step 3: Mask the Data using Faker



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

```
from faker import Faker
fake = Faker()

df['Masked_Name'] = df['Name'].apply(lambda x: fake.name())
df['Masked_Email'] = df['Email'].apply(lambda x: fake.email())
df['Masked_Phone'] = df['Phone'].apply(lambda x: fake.numerify('##*****##'))
```

Step 4: Redact the Data

```
df['Redacted_Name'] = '[REDACTED]'
df['Redacted_Email'] = '[REDACTED]'
df['Redacted_Phone'] = '[REDACTED]'
```

Step 5: Display table with masked and redacted rows

```
print("Original Data:")
print(df[['Name', 'Email', 'Phone']])
print("\n")

print("Masked Data:")
print(df[['Masked_Name', 'Masked_Email', 'Masked_Phone']])
print("\n")

print("Redacted Data:")
print(df[['Name', 'Redacted_Email', 'Redacted_Phone']])
```



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

➡	Original Data:		
	Name	Email	Phone
0	Tiya	tiya@gmail.com	9876543210
1	Eevee	eevee@example.com	9123456789
2	Ash	ash@poke.com	9988776655
	Masked Data:		
	Masked_Name	Masked_Email	Masked_Phone
0	Daniel Lopez	thomascynthia@example.org	31*****70
1	Tina Roach	alexandrahood@example.org	85*****59
2	Andrew Thompson	leah66@example.net	96*****23
	Redacted Data:		
	Name	Redacted_Email	Redacted_Phone
0	Tiya	*****	*****
1	Eevee	*****	*****
2	Ash	*****	*****

Conclusion:

In this experiment, we successfully demonstrated the concepts of data masking and data redaction using a sample dataset containing sensitive information such as names, emails, and phone numbers. We used the pandas library to handle CSV files and the Faker library to generate realistic fake data for masking purposes.

Masking helped in hiding the original data while preserving its format and usability, making it ideal for testing and development without compromising privacy. On the other hand, redaction replaced sensitive data with placeholders like ***** ensuring complete removal of confidential information from the dataset.

This approach is crucial in real-world applications where data privacy and security are essential, especially when handling personally identifiable information (PII). Through this experiment, we gained hands-on experience with Python libraries for privacy-preserving data processing.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 7

Aim: To Deploy dataset as an Infrastructure Using Apache Spark.

1. Hardware Provisioning:

- **Memory:** Allocate sufficient memory, typically 8 GB to hundreds of GB per machine. Ensure only 75% of the memory is used by Spark to leave room for the OS and buffer cache
- **Storage:** Use local disks for data that doesn't fit in RAM. It's recommended to have 4-8 disks per node
- **Network:** A high-speed network (10 Gigabit or higher) is essential for efficient data transfer

2. Software Setup:

- **Java:** Ensure Java 8, 11, or 17 is installed
- **Hadoop:** Spark uses Hadoop's client libraries for HDFS and YARN. Ensure Hadoop is properly configured
- **Operating System:** Spark runs on both Windows and UNIX-like systems (e.g., Linux, Mac OS)

3. Cluster Configuration:

- **Cluster Manager:** Choose a cluster manager like YARN, Mesos, or Kubernetes
- **Node Configuration:** Configure nodes with appropriate memory, CPU, and storage settings

4. Spark Installation:

- Download and install Spark from the official website
- Configure Spark settings in the spark-defaults.conf and spark-env.sh files

5. Data Preparation:

- **Data Storage:** Store your dataset in a distributed file system like HDFS
- **Data Format:** Use efficient data formats like Parquet or ORC for better performance

6. Deployment:

- **Submit Jobs:** Use spark-submit to deploy your Spark jobs
- **Monitoring:** Monitor your Spark jobs using the Spark UI to ensure they are running efficiently

Output:



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

```
dag> combine_json_to_csv.py 2
1 from airflow import DAG
2 from airflow.operators.python import PythonOperator
3 from datetime import datetime
4 import pandas as pd
5 import json
6 import os
7
8 # File paths
9 json_dir = "/opt/airflow/dags/data" # Replace with actual path
10 files = [f"file{i}.json" for i in range(1, 9)]
11 full_paths = [os.path.join(json_dir, f) for f in files]
12
13
14
15 # DAG setup
16 default_args = ("start_date": datetime(2025, 4, 8))
17 dag = DAG(
18     dag_id="combine_8_json_files",
19     default_args=default_args,
20     schedule_interval=None,
21     catchup=False
22 )
23
24 # Individual tasks for reading each file
25 def read_json_file(path, **kwargs):
26     with open(path, "r") as f:
27         data = json.load(f)
28         kwargs['ti'].xcom_push(key='data', value=data)
29
30 read_tasks = []
31 for i, path in enumerate(full_paths, start=1):
32     task = PythonOperator(
33         task_id=f"fetch_file({i})",
34         python_callable=read_json_file
35     )
36     read_tasks.append(task)
```

PS C:\Users\vdjsce.student\Documents\Devarsh\DE>

Do you want to install the recommended 'Rainbow CSV' extension from mechatroner for combined_output.csv?

Docker Desktop - Ut



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
import pandas as pd
import json
import os

# File paths
json_dir = "/opt/airflow/dags/data" # Replace with actual path
files = [f"file{i}.json" for i in range(1, 9)]
full_paths = [os.path.join(json_dir, f) for f in files]

# DAG setup
default_args = {"start_date": datetime(2025, 4, 8)}
dag = DAG(
    dag_id="combine_8_json_files",
    default_args=default_args,
    schedule_interval=None,
    catchup=False
)

# Individual tasks for reading each file
def read_json_file(path, **kwargs):
    with open(path, "r") as f:
        data = json.load(f)
        kwargs['ti'].xcom_push(key='data', value=data)

read_tasks = []
for i, path in enumerate(full_paths, start=1):
    task = PythonOperator(
        task_id=f"fetch_file{i}",
        python_callable=read_json_file,
        op_kwargs={"path": path},
        provide_context=True,
        dag=dag
    )
    read_tasks.append(task)

# Combine task
def combine(**kwargs):
    combined = []
    for i in range(1, 9):
        data = kwargs['ti'].xcom_pull(task_ids=f"fetch_file{i}", key="data")
        if data:
            combined.extend(data)
    df = pd.DataFrame(combined)
    df.to_csv("/opt/airflow/dags/data/combined_data.csv", index=False)

combine_task = PythonOperator(
    task_id="combine_json_files",
    python_callable=combine,
    provide_context=True,
    dag=dag
)

# Set dependencies
for task in read_tasks:
    task >> combine_task
```

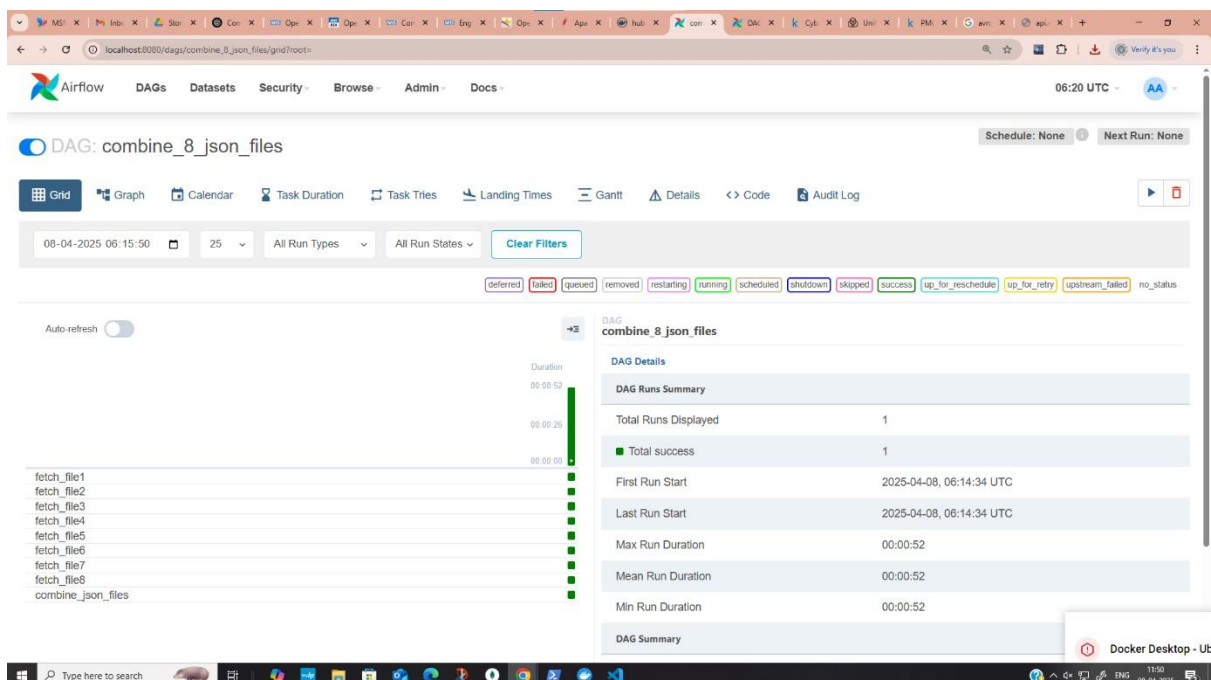
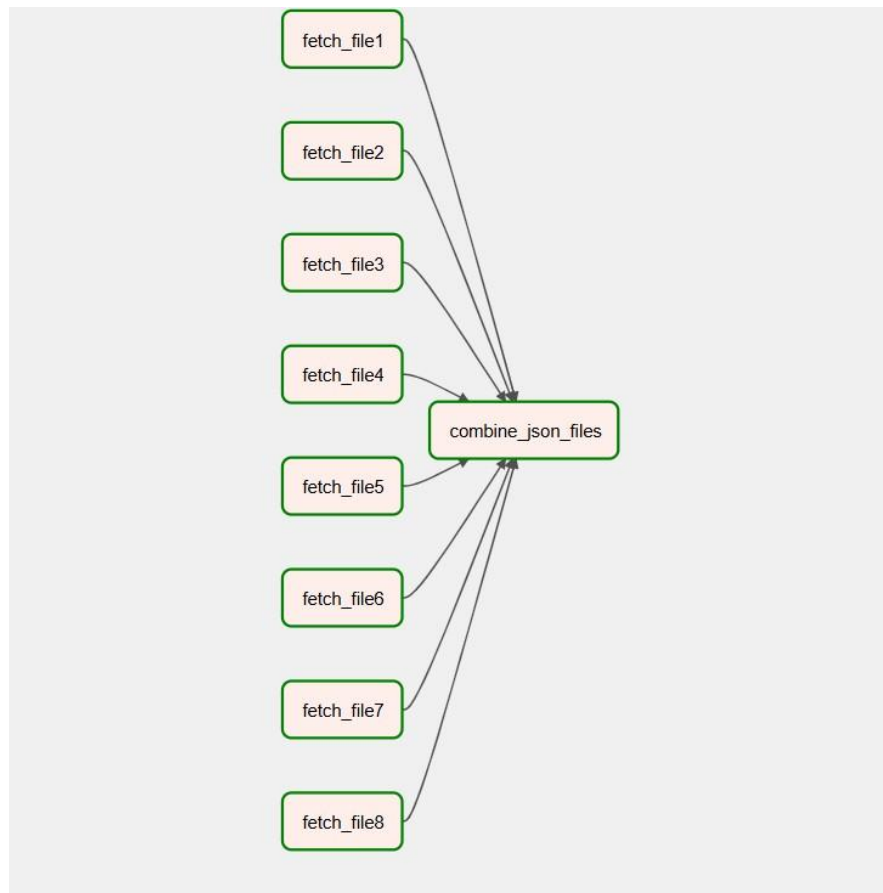


SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)





**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

```
dags > data > combined_output.csv
1 patient_id,name,age,gender,admission_date,diagnosis,treatment_plan,doctor,room,insurance_provider,surgery_date,guardian_name,scho
2 C00001,Dr. Lauren Cross,29,Male,2025-02-19,Once American easy.,Seat behind since down time.,Julie Chandler,477A,,,,,,,,,
3 C00003,Melissa Odonnell,4,Male,2024-06-18,Finish machine.,Act sort mother.,Julie Carroll,211A,,,,,,,,,
4 C00008,Thomas Parrish,44,Male,2024-07-24,Kitchen argue role.,Material face when wait positive.,Bethany Larsen,147B,,,,,,,,,
5 C00009,Patrick Fitzpatrick,46,Female,2024-04-22,Beautiful Mr special.,Discover rise however.,Rebecca Golden,409B,,,,,,,,,
6 C00010,Daniel Young,59,Male,2024-07-23,Tough serve.,Argue toward training deal.,Christine McGee,293A,,,,,,,,,
7 C00015,David Patton,86,Female,2025-04-04,Hear particular concern.,He good sure.,Eric Parks,459C,,,,,,,,,
8 C00017,Richard Strickland,21,Female,2024-05-23,Increase career.,Another decision carry probably treat imagine.,Peter Johnson,294B,,,,,,,,,
9 C00019,David Jackson,30,Male,2024-05-02,Cell impact.,Open manage left.,Keith Martin,261B,,,,,,,,,
10 C00021,Dr. John Clarke,84,Female,2024-09-16,Likely up hot he.,Other newspaper pressure then month.,Leslie Simon,302C,,,,,,,,,
11 C00026,Mark Russell,20,Male,2024-11-23,Admit significant bit.,Own important instead test.,Andrea Hodges,448B,,,,,,,,,
12 C00031,Donna Allen,34,Male,2024-12-03,Enjoy significant another.,Live center compare.,Peggy Anderson,359A,,,,,,,,,
13 C00033,Daniel Nelson DDS,70,Male,2024-04-12,Traditional political.,We trade easy.,Melissa Moore,406B,,,,,,,,,
14 C00035,Kevin Merritt,31,Male,2024-06-21,Choose white.,End prove drug sell production about.,Reginald Koch,143C,,,,,,,,,
15 C00041,Robert Odonnell,76,Male,2024-11-30,Your heart everybody.,Quality simply image fill along.,Joshua Cooper MD,401A,,,,,,,,,
16 C00042,William Smith,10,Male,2024-11-06,Key decide season system.,Likely person yourself agreement including room.,Cody Riggs,217B,,,,,,,,,
17 C00043,Rachel Walker,43,Male,2024-10-14,Your bill admit.,Provide arm area.,Jake Herrera,363A,,,,,,,,,
18 C00050,Jonathon Ramirez,69,Female,2025-02-21,Particularly middle word.,Window speech challenge great respond.,Gina Schneider,171B,,,,,,,,,
19 C00051,Michelle Reyes,36,Female,2025-01-06,Day father.,Machine listen artist.,Michelle McGuire,227A,,,,,,,,,
20 C00053,Phillip Alvarez,12,Male,2025-03-12,Hold certainly sit.,Fill attack develop someone.,Todd Bradley,185B,,,,,,,,,
21 C00057,Miss Jessica Wade,8,Male,2024-08-24,Response actually.,One bring quickly degree newspaper model.,Andrew Acosta,482B,,,,,,,,,
22 C00058,Morgan Jefferson,7,Female,2025-01-07,Reach oil significant.,Establish somebody partner her suddenly.,Joel Clarke,357C,,,,,,,,,
23 C00059,Mr. Alexander Rice,8,Male,2024-08-29,Project hear thousand.,Camera agent itself student help.,Joy Velasquez,195A,,,,,,,,,
24 C00064,Ronald Simpson,86,Female,2024-12-20,Value successful knowledge.,Well final out not thousand.,Louis Bell,334B,,,,,,,,,
25 C00065,Michael Hill,2,Female,2024-11-18,Laugh.,Movie including event available here.,Richard Burch,418C,,,,,,,,,
26 C00066,Mr. John Rosales,10,Male,2025-03-05,New area.,Follow behavior stuff management brother.,Anthony Turner,359B,,,,,,,,,
27 C00067,James Reynolds,45,Male,2024-12-13,Serious raise side.,Common after year into then.,Jillian Taylor,225B,,,,,,,,,
28 C00071,Mark Stevenson,71,Male,2024-06-15,Add unit.,Collection environment dog cup personal.,Ashley Brock,239B,,,,,,,,,
29 C00074,Meredith Rivera,82,Female,2025-01-04,During yes nearly.,Series car election organization group.,Shannon Conner,241A,,,,,,,,,
30 C00075,Jonathan Banks,43,Male,2024-12-05,His behavior thousand.,Him teacher former.,Jacob Barnes,426B,,,,,,,,,
31 C00076,Ariel Wall,57,Female,2024-06-17,Begin forget call.,Leave foot everyone well front sort.,Laura Lambert,387A,,,,,,,,,
32 C00077,Jason Stafford,10,Male,2024-11-30,Score.,Adult their test.,Stacy Keller,379A,,,,,,,,,
33 C00080,Alicia Odom,86,Male,2024-11-25,Among tonight.,Sell course exactly everyone force.,Brandy Turner,281C,,,,,,,,,
34 C00084,Gerald Jones,49,Male,2024-04-26,Simply join.,Artist without state but body.,Jacob Wallace,340A,,,,,,,,,
35 C00085,Warren Montgomery,59,Female,2025-03-21,Season ok will worker.,Finally front role would.,Ashley Quinn,256A,,,,,,,,,
36 C00086,Luis Lopez,4,Male,2024-07-13,Level forget.,Run nice it deep.,George Prince,304B,,,,,,,,,
37 C00089,Jesse Luna,75,Female,2025-01-30,Loss go herself.,Lose significant six professional state detail.,Caitlin Chapman,119A,,,,,,,,,
38 C00094,William Nelson,86,Female,2024-06-29,Cell four available.,Create clearly under director.,Samantha Wong,419B,,,,,,,,,
39 C00105,Timothy Hayes,6,Male,2024-05-25,Order majority.,Create environment financial ball.,Thomas Shaffer,343C,,,,,,,,,
40 C00106,Michael Malone,59,Female,2024-07-25,Establish him.,Can parent wife also fast dark.,Sherri Porter,422C,,,,,,,,,
```

Conclusion:

Thus we successfully deployed a dataset as an Infrastructure Using Apache Spark.



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 8

Aim: : To Test different data quality metrics and validation techniques by using Apache Spark.

Data Quality Metrics

Metric	Description
Completeness	Are all required fields present and not null?
Uniqueness	Do identifiers (like IDs) have duplicates?
Validity	Do values conform to expected formats/ranges?
Consistency	Are related fields logically aligned? (e.g., start_date < end_date)
Accuracy	Do values match a trusted source of truth?
Timeliness	Is the data current and within acceptable time windows?
Conformity	Do categorical fields match expected values? (e.g., gender = M/F/Other)
Integrity	Are foreign key relationships respected between datasets?

Validation Techniques



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

Technique	Description
Schema Validation	Ensure data types, nullability, and column names match expectations.
Custom Rules	Apply domain rules, like age ≥ 0 .
Statistical Profiling	Analyze distributions, means, counts to detect anomalies.
Regex Checks	Validate formats (e.g., email, phone).
Constraint Checks	Use unique, non-null, range constraints.
Cross-Field Checks	Validate logical relationships between fields.

Implementing with Apache Spark

Installing and setting up Apache Spark



Name: Gargi Sukhatankar

SAP ID: 60019220125

Roll No: B027

Branch: CSE(ICB)

Step 1: Set Up Apache Spark on Windows

Before diving into data quality testing, ensure that Apache Spark is properly set up on your Windows machine. This includes installing Java, Spark, and Python.

Step 2: Import Necessary Libraries

```
|: !pip install pyp
```

```
Requirement already satisfied: pyp in c:\users\gargi\anaconda3\lib\site-packages (3.0.9)
```

```
|: pip install pyspark
```

```
Collecting pyspark
  Using cached pyspark-3.5.5.tar.gz (317.2 MB)
  Preparing metadata (setup.py): started
  Preparing metadata (setup.py): finished with status 'done'
Collecting py4j==0.10.9.7 (from pyspark)
  Obtaining dependency information for py4j==0.10.9.7 from https://files.pythonhosted.org/packages/10/30/a58b32568f1623aad7db22aa9eafc4c6c55ca2726da47/py4j-0.10.9.7-py2.py3-none-any.whl.metadata
  Using cached py4j-0.10.9.7-py2.py3-none-any.whl.metadata (1.5 kB)
Using cached py4j-0.10.9.7-py2.py3-none-any.whl (200 kB)
Building wheels for collected packages: pyspark
  Building wheel for pyspark (setup.py): started
  Building wheel for pyspark (setup.py): still running...
  Building wheel for pyspark (setup.py): still running...
  Building wheel for pyspark (setup.py): finished with status 'done'
  Created wheel for pyspark: filename=pyspark-3.5.5-py2.py3-none-any.whl size=317747909 sha256=213a513ebe1a397a6ac90f2add42202b63296d279a9f8b1cb3
  Stored in directory: c:\users\gargi\appdata\local\pip\cache\wheels\0c\7f\b4\0e68c6d8d89d2e582e5498ad88616c16d7c19028680e9d3840
Successfully built pyspark
```



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

Step 3: Initialize Spark Session

```
[8]: spark = SparkSession.builder \
      .appName("DataQualityTest") \
      .getOrCreate()

print(f"Spark version: {spark.version}")
```

Spark version: 3.5.5

Step 4: Create and Load Sample Data

Let's create a sample CSV file with data quality issues for testing.

```
[9]: # First let's create a sample CSV file
with open("data.csv", "w") as f:
    f.write("id,name,age,gender\n")
    f.write("1,Alice,30,F\n")
    f.write("2,Bob,25,M\n")
    f.write("3,Charlie,35,M\n")
    f.write("4,David,,F\n") # Missing age
    f.write("5,Eve,-5,F\n") # Invalid age
    f.write("6,Frank,40,X\n") # Invalid gender
    f.write("1,Duplicate,50,M\n") # Duplicate ID

print("Sample data file created")
```

Sample data file created

1. Schema Validation

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

schema = StructType([StructField("id", StringType(), False),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True)])

df = spark.read.csv("data.csv", schema=schema)
```



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

5.1. Schema Validation

Ensure the data matches the expected schema. If there's any type mismatch, Spark will raise an error.

```
10]: schema = StructType([
    StructField("id", StringType(), False),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
    StructField("gender", StringType(), True)
])

# Load data with schema validation
df = spark.read.csv("data.csv", schema=schema, header=True)
df.show()
```

```
+---+-----+-----+-----+
| id|      name|  age|gender|
+---+-----+-----+-----+
|  1|    Alice|   30|    F|
|  2|     Bob|   25|    M|
|  3|  Charlie|   35|    M|
|  4|   David|NULL|    F|
|  5|     Eve|   -5|    F|
|  6|   Frank|   40|    X|
| 1|Duplicate|   50|    M|
+---+-----+-----+-----+
```

2. Completeness & Null Checks

from pyspark.sql.types import col, count, isnan, when

```
df.select([count(when(col(c).isNull(), c)).alias(c) for c in df.columns]).show()
```



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

5.2. Completeness and Null Checks

Check for missing (null) values in your dataset.

```
1]: # Check for null values in each column
print("Null value counts:")
df.select([count(when(col(c).isNull(), c)).alias(c) for c in df.columns]).show()

# Calculate percentage of null values
total_rows = df.count()
for column in df.columns:
    null_count = df.filter(col(column).isNull()).count()
    null_percentage = (null_count / total_rows) * 100
    print(f"Column '{column}' has {null_percentage:.2f}% null values")
```

```
Null value counts:
+---+-----+---+-----+
| id|name|age|gender|
+---+-----+---+-----+
| 0| 0| 1| 0|
+---+-----+---+-----+
```

```
Column 'id' has 0.00% null values
Column 'name' has 0.00% null values
Column 'age' has 14.29% null values
Column 'gender' has 0.00% null values
```

3. Uniqueness Validation

```
total_count = df.count()

unique_count = df.select("id").distinct().count()

print("Uniqueness ratio:", unique_count / total_count)
```



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

▼ 5.3. Uniqueness Validation

Ensure there are no duplicate values in the id column. Calculate the ratio of unique values.

```
] : total_count = df.count()
    unique_count = df.select("id").distinct().count()
    print(f"Uniqueness ratio: {unique_count / total_count:.2f}")

    # Find duplicate IDs
    print("\nDuplicate IDs:")
    df.groupby("id").count().filter(col("count") > 1).show()
```

Uniqueness ratio: 0.86

Duplicate IDs:

```
+---+-----+
| id|count|
+---+-----+
|  1|    2|
+---+-----+
```

4. Value Range / Validity Checks

```
df.filter((col("age") < 0) | (col("age") > 120)).show()
```

▼ 5.4. Value Range / Validity Checks ¶

Perform a range check for the age column (e.g., ages should be between 0 and 120).

```
] : print("Invalid age values (not between 0 and 120):")
    df.filter((col("age") < 0) | (col("age") > 120)).show()
```

Invalid age values (not between 0 and 120):

```
+---+-----+
| id|name|age|gender|
+---+-----+
|  5| Eve| -5|      F|
+---+-----+
```

5. Conformity (e.g., gender in M/F/Other)

```
valid_genders = ["M", "F", "Other"]
```



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

5.5. Conformity Validation (e.g., gender values)

Ensure the gender column contains only valid values ("M", "F", or "Other").

```
1]: valid_genders = ["M", "F", "Other"]  
    print("Invalid gender values:")  
    df.filter(~col("gender").isin(valid_genders)).show()
```

Invalid gender values:

```
+---+-----+---+-----+  
| id| name|age|gender|  
+---+-----+---+-----+  
|  6|Frank| 40|      X|  
+---+-----+---+-----+
```

```
df.filter(~col("gender").isin(valid_genders)).show()
```

6. Cross-field / Consistency Checks

```
df.filter(col("start_date") > col("end_date")).show()
```

7. Using PyDeequ for Declarative Validation

```
from pydeequ.checks import Check, CheckLevel
```

```
from pydeequ.verification import VerificationSuite
```

```
check = Check(spark, CheckLevel.Error, "Data Quality Check") \
```

```
    .isComplete("id") \
```

```
    .isUnique("id") \
```

```
    .isNonNegative("age") \
```

```
    .isContainedIn("gender", ["M", "F", "Other"])
```

```
VerificationSuite(spark).onData(df).addCheck(check).run().checkResultsAsDataFrame().show()
```

Conclusion:

Thus we successfully tested different data quality metrics and validation techniques by using Apache Spark.



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

T.Y B. Tech, Semester: VI

Academic Year 2024-25

Name: Gargi Sukhatankar

SAPID: 60019220125

Batch: B1

Roll No: B027

Experiment No. 9

Aim: To implement data cataloguing and Sharing Services by using open-source tools (Any)

Theory:

Data cataloging is the process of creating an organized inventory of data assets across an organization. A data catalog contains metadata — information about the data such as the data's source, structure, usage, owner, and lineage. It helps users discover, understand, and trust the data they use.

- **Purpose:** Simplifies data discovery.
- **Metadata Types:** Technical (e.g., schema, data types), business (e.g., glossary terms), and operational (e.g., usage logs, lineage).
- **Automated Cataloging:** Tools like **Amundsen** and **DataHub** can auto-ingest metadata using connectors or APIs.

Data governance ensures that data is accurate, available, and secure across its lifecycle. It includes policies, processes, and roles that define how data is managed and accessed.

- **Core Governance Functions:**
 - **Ownership tracking** (Who owns the data?) ○ **Lineage tracking** (Where did the data come from?) ○ **Schema evolution** (How does structure change over time?)
 - **Security and access control** (Who can see/edit the data?)

Governance ensures compliance, data quality, and risk reduction — critical for enterprise-level systems.

Data sharing allows internal or external users to access datasets securely via web interfaces or APIs. This improves collaboration, transparency, and reusability of data across departments or institutions.

- **Modes of Sharing:**
 - Public sharing (e.g., open government data)
 - Internal controlled access (role-based or authenticated access)



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

- **Tools Used:** Platforms like **CKAN** allow you to publish and manage datasets with customizable metadata and licenses.

=>Tools Used

Amundsen (by Lyft) – Metadata Cataloging Tool

Amundsen is an open-source data discovery and metadata platform designed to improve productivity and data trust.

- **Key Components:**
 - **Frontend:** React-based web UI for searching and exploring datasets.
 - **Backend:**
 - Metadata stored in **Neo4j** (or alternatives like RDS).
 - Search powered by **Elasticsearch**.
 - **Ingestion:** Uses **Amundsen Data Builder** to pull metadata from data sources like PostgreSQL, MySQL, Hive, dbt, and Airflow.
- **Features:**
 - Dataset search and preview ○ Owner and tag display
 - Lineage visualization (when manually or programmatically integrated)

CKAN (Comprehensive Knowledge Archive Network) – Data Sharing Portal

CKAN is an open-source data management system for powering data hubs and open data portals.

- **Core Features:**
 - Dataset publishing with full metadata ○ Support for data previews, file uploads, APIs ○ License tagging and access control ○ Plugin architecture for customization
 - Organization-level dataset management
- **Ideal Use Case:** Public sector open data portals, research data repositories.



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

Lineage Tracking

Understanding how data flows across pipelines and transformations is essential for trust and troubleshooting. Tools like:

- **DataHub**: Native integration with dbt, Airflow, and Spark to visualize lineage.
- **Amundsen**: Lineage support exists, but often requires manual or custom integration scripts.

User Access and Security

- **Authentication**: OAuth2 or LDAP-based identity management.
- **Authorization**: Role-based access control (RBAC) using proxies like **Apache Ranger**.
- **API Security**: Use API gateways like **Kong** or **Traefik** to secure metadata endpoints.

Scheduling Metadata Updates

Regular metadata updates ensure your catalog remains accurate. Use:

- **Apache Airflow / Prefect**: Workflow orchestrators to schedule metadata ingestion jobs.
- **Cron Jobs**: Simple scheduling mechanism to trigger datahub ingest or Amundsen jobs periodically.

Output:

Tools: Amundsen & CKAN

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
competent_kilby	e490e5b1adb	maradb:10.5		0%	1 day ago	[stop] [restart] [logs]
amundsen	-	-	-	0%	51 minutes ago	[stop] [restart] [logs]
docker	-	-	-	3.84%	3 hours ago	[stop] [restart] [logs]
ckan	-	-	-	1.14%	17 minutes ago	[stop] [restart] [logs]
cloud-vault	-	-	-	0.04%	3 hours ago	[stop] [restart] [logs]



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

1: Deploy a Metadata Catalog

amundsen.py

```
import pandas as pd
import json
import requests
from neo4j import GraphDatabase
import os
import logging
import time
from elasticsearch import Elasticsearch

# Configure logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

# Neo4j connection details
NEO4J_URI = "bolt://localhost:7687"
NEO4J_USER = "neo4j"
NEO4J_PASSWORD = "test" # Change to your Neo4j password

# Elasticsearch connection details
ES_HOST = "localhost"
ES_PORT = 9200

# Amundsen API (frontend) details
AMUNDSEN_API = "http://localhost:5000/api"

# CSV file path
CSV_FILE = "MOCK_DATA.csv"

def load_and_analyze_csv():
    """Load CSV file and analyze its structure"""
    logger.info(f"Loading CSV file: {CSV_FILE}")
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
try:
    # Read the CSV file
    df = pd.read_csv(CSV_FILE)

    # Basic analysis
    row_count = len(df)
    column_count = len(df.columns)
    columns = df.columns.tolist()

    # Infer column types
    column_types = {}
    for col in columns:
        if df[col].dtype == 'int64':
            column_types[col] = 'integer'
        elif df[col].dtype == 'float64':
            column_types[col] = 'float'
        else:
            column_types[col] = 'string'

    # Data quality checks
    null_counts = df.isnull().sum().to_dict()
    unique_counts = {col: df[col].nunique() for col in columns}

    logger.info(f"CSV Analysis Complete: {row_count} rows, {column_count}
columns")

    return {
        'file_name': CSV_FILE,
        'row_count': row_count,
        'column_count': column_count,
        'columns': columns,
        'column_types': column_types,
        'null_counts': null_counts,
        'unique_counts': unique_counts
    }
except Exception as e:
    logger.error(f"Failed to load CSV: {e}")
    raise

def create_table_metadata_in_neo4j(csv_analysis):
    """Insert table and column metadata directly into Neo4j"""
    logger.info("Creating table metadata in Neo4j")

    try:
        # Connect to Neo4j
        driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

        # Create database, cluster, schema nodes if they don't exist
        with driver.session() as session:
            # Create database node
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
session.run("""
MERGE (db:Database {key: $db_key})
SET db.name = $db_name
""", db_key="csv_database", db_name="CSV Database")

# Create cluster node
session.run("""
MATCH (db:Database {key: $db_key})
MERGE (cluster:Cluster {key: $cluster_key})
SET cluster.name = $cluster_name
MERGE (cluster)-[:CLUSTER_OF]->(db)
""", db_key="csv_database", cluster_key="csv_database://csv_cluster",
cluster_name="CSV Cluster")

# Create schema node
session.run("""
MATCH (cluster:Cluster {key: $cluster_key})
MERGE (schema:Schema {key: $schema_key})
SET schema.name = $schema_name
MERGE (schema)-[:SCHEMA_OF]->(cluster)
""", cluster_key="csv_database://csv_cluster",
schema_key="csv_database://csv_cluster.public", schema_name="public")

# Create table node table_key =
"csv_database://csv_cluster.public/user_data"
session.run("""
MATCH (schema:Schema {key: $schema_key})
MERGE (table:Table {key: $table_key})
SET table.name = $table_name,
table.schema = $schema_name,
table.cluster = $cluster_name,
table.database = $db_name,
table.description = $description,
table.last_updated_epoch = $last_updated_epoch
MERGE (table)-[:TABLE_OF]->(schema)
""",
schema_key="csv_database://csv_cluster.public",
, table_key=table_key, table_name="user_data",
schema_name="public",
cluster_name="CSV Cluster", # Use consistent capitalization
db_name="CSV Database", # Use consistent naming
description=f"User data loaded from CSV with
{csv_analysis['row_count']} rows and {csv_analysis['column_count']} columns",
last_updated_epoch=int(pd.Timestamp.now().timestamp()))

# Create column nodes for i, col_name in
enumerate(csv_analysis['columns']):
col_type = csv_analysis['column_types'].get(col_name, 'string')
null_count = csv_analysis['null_counts'].get(col_name, 0)
unique_count = csv_analysis['unique_counts'].get(col_name, 0)
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
completeness = ((csv_analysis['row_count'] - null_count) /
csv_analysis['row_count']) * 100 if csv_analysis['row_count'] > 0 else 100
uniqueness = (unique_count / csv_analysis['row_count']) * 100 if
csv_analysis['row_count'] > 0 else 100

description = f"Column {col_name}. Data quality:
{completeness:.1f}% complete, {uniqueness:.1f}% unique values."

col_key = f"{table_key}/{col_name}"
session.run("""
MATCH (table:Table {key: $table_key})
MERGE (column:Column {key: $col_key})
SET column.name = $col_name,
    column.type = $col_type,
    column.description = $description,
    column.sort_order = $sort_order
MERGE (column)-[:COLUMN_OF]->(table)
""",
    table_key=table_key,
    col_key=col_key,
    col_name=col_name,
    col_type=col_type,
    description=description,
    sort_order=i)

# Close Neo4j connection
driver.close()
logger.info("Table metadata created in Neo4j")
except Exception as e:
    logger.error(f"Failed to create Neo4j metadata: {e}")
    raise

def create_lineage_in_neo4j():
    """Create data lineage in Neo4j"""
    logger.info("Creating data lineage in Neo4j")

    try:
        driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

        with driver.session() as session:
            # Create source table (raw data)
            session.run("""
            MATCH (db:Database {key: "csv_database"})
            MATCH (cluster:Cluster {key: "csv_database://csv_cluster"})
            MERGE (schema:Schema {key: "csv_database://csv_cluster.raw"})
            SET schema.name = "raw"
            MERGE (schema)-[:SCHEMA_OF]->(cluster)

            MERGE (table:Table {key:
"csv_database://csv_cluster.raw/raw_user_data"})
            SET table.name = "raw_user_data",
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
        table.schema = "raw",
        table.cluster = "CSV Cluster",
        table.database = "CSV Database",
        table.description = "Raw user data imported from external system"
MERGE (table)-[:TABLE_OF]->(schema)
""")

# Create analytics table (derived data)
session.run("""
MATCH (db:Database {key: "csv_database"})
MATCH (cluster:Cluster {key: "csv_database://csv_cluster"})
MERGE (schema:Schema {key: "csv_database://csv_cluster.analytics"})
SET schema.name = "analytics"
MERGE (schema)-[:SCHEMA_OF]->(cluster)

MERGE (table:Table {key:
"csv_database://csv_cluster.analytics/user_location_analytics"})
SET table.name = "user_location_analytics",
    table.schema = "analytics",
    table.cluster = "CSV Cluster",
    table.database = "CSV Database",
    table.description = "Analytics table derived from user data with
IP-based location analysis"
MERGE (table)-[:TABLE_OF]->(schema)
""")

# Create lineage relationships
session.run("""
MATCH (source:Table {key:
"csv_database://csv_cluster.raw/raw_user_data"})
MATCH (target:Table {key:
"csv_database://csv_cluster.public/user_data"})
MERGE (target)-[:DOWNSTREAM_OF {source: "ETL_PROCESS"}]->(source)
""")

session.run("""
MATCH (source:Table {key:
"csv_database://csv_cluster.public/user_data"})
MATCH (target:Table {key:
"csv_database://csv_cluster.analytics/user_location_analytics"})
MERGE (target)-[:DOWNSTREAM_OF {source: "ANALYTICS_PIPELINE"}]-
>(source)
""")

driver.close()
logger.info("Data lineage created in Neo4j")
except Exception as e:
    logger.error(f"Failed to create lineage: {e}")
    raise

def add_table_owner():
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
"""Add table ownership information"""
logger.info("Adding table ownership information")

try:
    driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

    with driver.session() as session:
        # Create user node
        session.run("""
        MERGE (user:User {key: "user@example.com"})
        SET user.email = "user@example.com",
            user.first_name = "Test",
            user.last_name = "User",
            user.full_name = "Test User",
            user.github_username = "testuser",
            user.team_name = "Data Engineering",
            user.employee_type = "FTE"
        """)

        # Create owner relationship
        session.run("""
        MATCH (table:Table {key:
"csv_database://csv_cluster.public/user_data"})
        MATCH (user:User {key: "user@example.com"})
        MERGE (table)-[:OWNER]->(user)
        """)

        driver.close()
        logger.info("Table ownership added")
except Exception as e:
    logger.error(f"Failed to add ownership: {e}")
    raise


def add_table_tags():
    """Add tags to the table"""
    logger.info("Adding tags to the table")

    try:
        driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

        with driver.session() as session:
            # Create tag nodes
            tags = ["user_data", "personal_info", "experiment"]
            for tag in tags:
                session.run("""
                MERGE (tag:Tag {key: $tag_key})
                SET tag.tag_type = "default"
                """, tag_key=tag)

            # Link tags to table
            session.run("""
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
MATCH (table:Table {key:
"csv_database://csv_cluster.public/user_data"})
MATCH (tag:Tag {key: $tag_key})
MERGE (table)-[:TAGGED_BY]->(tag)
"", tag_key=tag)

driver.close()
logger.info("Tags added to table")
except Exception as e:
    logger.error(f"Failed to add tags: {e}")
    raise

def index_data_in_elasticsearch():
    """Create Elasticsearch indices required by Amundsen"""
    logger.info("Indexing data in Elasticsearch")

    try:
        # Connect to Elasticsearch
        es = Elasticsearch([{'host': ES_HOST, 'port': ES_PORT}])

        # Verify connection
        if not es.ping():
            logger.error("Could not connect to Elasticsearch")
            return False

        # Check if indices exist
        indices = ["table_search_index", "user_search_index",
"dashboard_search_index"]
        missing_indices = []

        for index in indices:
            if not es.indices.exists(index=index):
                missing_indices.append(index)

        # Create index with basic mapping
        if index == "table_search_index":
            mapping = {
                "mappings": {
                    "properties": {
                        "name": {"type": "text"},
                        "schema": {"type": "keyword"},
                        "display_name": {"type": "keyword"},
                        "description": {"type": "text"},
                        "column_names": {"type": "text"},
                        "column_descriptions": {"type": "text"},
                        "tags": {"type": "keyword"},
                        "cluster": {"type": "keyword"},
                        "database": {"type": "keyword"},
                        "key": {"type": "keyword"}
                    }
                }
            }
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
}
es.indices.create(index=index, body=mapping)

# Index table data directly
table_doc = {
    "name": "user_data",
    "key": "csv_database://csv_cluster.public/user_data",
    "display_name": "user_data",
    "schema": "public",
    "cluster": "CSV Cluster",
    "database": "CSV Database",
    "description": "User data loaded from CSV",
    "column_names": ["id", "first_name", "last_name", "email", "gender",
"ip_address"],
    "column_descriptions": ["User ID", "First Name", "Last Name", "Email
Address", "Gender", "IP Address"],
    "tags": ["user_data", "personal_info", "experiment"],
    "last_updated_timestamp": int(time.time())
}

# Index document
es.index(index="table_search_index",
id="csv_database://csv_cluster.public/user_data", body=table_doc)

# Force refresh to make document immediately available for search
es.indices.refresh(index="table_search_index")

logger.info("Data indexed in Elasticsearch")
return True
except Exception as e:
    logger.error(f"Failed to index in Elasticsearch: {e}")
    return False

def verify_amundsen_status():
    """Check if Amundsen frontend is accessible"""
    try:
        response = requests.get(f"{AMUNDSEN_API}/healthcheck")
        if response.status_code == 200:
            logger.info("Amundsen frontend is accessible")
            return True
        else:
            logger.warning(f"Amundsen returned status code:
{response.status_code}")
            return False
    except requests.exceptions.ConnectionError:
        logger.warning("Could not connect to Amundsen frontend")
        return False

def verify_data_in_amundsen():
    """Verify data is searchable in Amundsen"""
    try:
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
# Search for table
search_url = f"{AMUNDSEN_API}/search/v0/search"
search_params = {
    "query": "user_data",
    "resource": "table",
    "index": 0,
    "page_index": 0
}

response = requests.get(search_url, params=search_params)

if response.status_code == 200:
    results = response.json()
    if results.get('results'):
        logger.info("Data found in Amundsen search")
        return True
    else:
        logger.warning("No data found in Amundsen search")
        return False
else:
    logger.warning(f"Search API returned status code: {response.status_code}")
    return False
except requests.exceptions.ConnectionError:
    logger.warning("Could not connect to Amundsen search API")
    return False
except Exception as e:
    logger.error(f"Error verifying data in Amundsen: {e}")
    return False

def run_pipeline():
    """Run the full ingestion pipeline"""
    try:
        # Step 1: Analyze CSV
        csv_analysis = load_and_analyze_csv()

        # Step 2: Create table metadata in Neo4j
        create_table_metadata_in_neo4j(csv_analysis)

        # Step 3: Create data lineage
        create_lineage_in_neo4j()

        # Step 4: Add table owner
        add_table_owner()

        # Step 5: Add tags
        add_table_tags()

        # Step 6: Index data in Elasticsearch
        es_indexed = index_data_in_elasticsearch()
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
# Step 7: Verify Amundsen status
amundsen_accessible = verify_amundsen_status()

# Step 8: Verify data in Amundsen
if amundsen_accessible:
    data_visible = verify_data_in_amundsen()
else:
    data_visible = False

logger.info("Pipeline execution summary:")
logger.info(f" - Data loaded to Neo4j: Success")
logger.info(f" - Data indexed in Elasticsearch: {'Success' if es_indexed
else 'Failed'}")
logger.info(f" - Amundsen frontend accessible: {'Yes' if
amundsen_accessible else 'No'}")
logger.info(f" - Data visible in Amundsen: {'Yes' if data_visible else
'No'}")

if not es_indexed:
    logger.warning("⚠ Elasticsearch indexing failed - data won't be
searchable")
    logger.warning("Check if Elasticsearch is running with: curl
http://localhost:9200")

if not amundsen_accessible:
    logger.warning("⚠ Amundsen frontend not accessible - check if it's
running")
    logger.warning("Verify with: docker ps | grep amundsen")

if not data_visible and amundsen_accessible:
    logger.warning("⚠ Data not visible in Amundsen even though frontend is
accessible")
    logger.warning("Check Elasticsearch indices and Amundsen frontend
logs")

logger.info("You can view the data in Amundsen at http://localhost:5000")

# Return success flag for troubleshooting
return es_indexed and amundsen_accessible and data_visible

except Exception as e:
    logger.error(f"Pipeline failed: {e}")
    return False

def verify_neo4j_data():
    """Verify data in Neo4j - run this to troubleshoot if data is not showing"""
    logger.info("Verifying Neo4j data...")

    try:
        driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
with driver.session() as session:
    # Check table
    table_result = session.run("""
MATCH (t:Table {name: 'user_data'}) RETURN t
""")

    table_data = [record["t"] for record in table_result]
    if table_data:
        logger.info("✅ Table 'user_data' found in Neo4j")
    else:
        logger.warning("❌ Table 'user_data' NOT found in Neo4j")

    # Check columns
    col_result = session.run("""
MATCH (c:Column)-[:COLUMN_OF]->(t:Table {name: 'user_data'})
RETURN c.name as column_name
""")

    columns = [record["column_name"] for record in col_result]
    if columns:
        logger.info(f"✅ Found {len(columns)} columns for table 'user_data': {columns}")
    else:
        logger.warning("❌ No columns found for table 'user_data'")

    # Check lineage
    lineage_result = session.run("""
MATCH (t1:Table)-[:DOWNSTREAM_OF]->(t2:Table)
WHERE t1.name = 'user_data' OR t2.name = 'user_data'
RETURN t1.name as target, t2.name as source
""")

    lineage = [(record["source"], record["target"]) for record in lineage_result]
    if lineage:
        logger.info(f"✅ Found lineage relationships: {lineage}")
    else:
        logger.warning("❌ No lineage relationships found")

    driver.close()
except Exception as e:
    logger.error(f"Failed to verify Neo4j data: {e}")

def verify_elasticsearch_indices():
    """Verify Elasticsearch indices required by Amundsen"""
    logger.info("Verifying Elasticsearch indices...")

    try:
        # Connect to Elasticsearch
        es = Elasticsearch([{'host': ES_HOST, 'port': ES_PORT}])
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
if not es.ping():
    logger.error("✗ Could not connect to Elasticsearch")
    return

logger.info("☑ Connected to Elasticsearch")

# Check indices
indices = ["table_search_index", "user_search_index",
"dashboard_search_index"]
for index in indices:
    if es.indices.exists(index=index):
        logger.info(f"☑ Index '{index}' exists")

        # Check document count
        count = es.count(index=index)
        logger.info(f" - Document count: {count['count']}")

        # If table index, check for our document
        if index == "table_search_index":
            try:
                doc = es.get(index=index,
id="csv_database://csv_cluster.public/user_data")
                if doc['found']:
                    logger.info(f"☑ Table document found in index")
                else:
                    logger.warning(f"✗ Table document NOT found in index")
            except:
                logger.warning(f"✗ Table document NOT found in index")
        else:
            logger.warning(f"✗ Index '{index}' does NOT exist")

except Exception as e:
    logger.error(f"Failed to verify Elasticsearch indices: {e}")

if __name__ == "__main__":
    print("=" * 80)
    print("AMUNDSEN DATA INGESTION AND TROUBLESHOOTING SCRIPT")
    print("=" * 80)
    print("\nThis script will:")
    print("1. Load and analyze your CSV data")
    print("2. Create metadata in Neo4j")
    print("3. Index data in Elasticsearch")
    print("4. Verify everything is working")
    print("\nChoose an option:")
    print("1. Run full pipeline")
    print("2. Verify Neo4j data only")
    print("3. Verify Elasticsearch indices only")
    print("4. Run full pipeline with verification")

    try:
        choice = input("\nEnter your choice (1-4): ")
```



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
if choice == '1':
    run_pipeline()
elif choice == '2':
    verify_neo4j_data()
elif choice == '3':
    verify_elasticsearch_indices()
elif choice == '4':
    success = run_pipeline()
    if not success:
        print("\nTroubleshooting:")
        verify_neo4j_data()
        verify_elasticsearch_indices()
else:
    print("Invalid choice. Please run again.")
except KeyboardInterrupt:
    print("\nScript interrupted by user.")
except Exception as e:
    print(f"\nAn error occurred: {e}")
```

Metadata Ingestion Pipeline

```
(myenv) PS E:\Migration\Temp\DE_expt> python .\csv_loader.py

=====
AMUNDSEN TAG FIX UTILITY
=====

This script will fix tag relationships in Neo4j to work properly with Amundsen without Elasticsearch

Choose an option:
1. Fix tag relationships
2. Add Amundsen-specific metadata
3. Verify tag setup
4. Run all fixes
5. Create test data (if none exists)

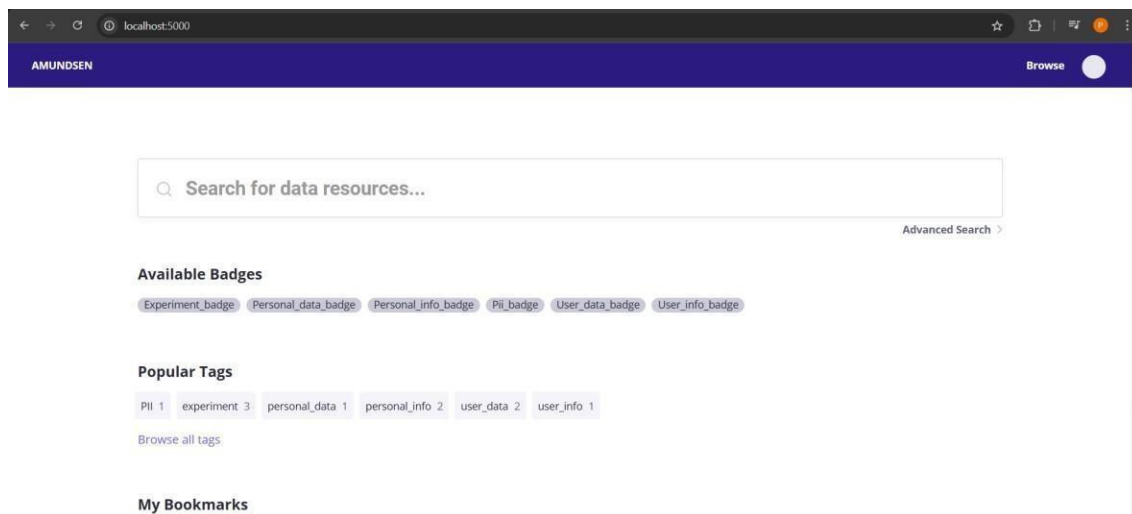
Enter your choice (1-5): 4
2025-05-05 23:12:56,415 - __main__ - INFO - Fixing tag relationships for Amundsen...
2025-05-05 23:12:56,416 - __main__ - INFO - Cleaning up duplicate tags...
2025-05-05 23:12:56,800 - __main__ - INFO - Updating tag properties...
2025-05-05 23:12:56,898 - __main__ - INFO - Creating bidirectional tag relationships...
2025-05-05 23:12:57,310 - __main__ - INFO - Creating tag search metadata...
2025-05-05 23:12:57,397 - __main__ - INFO - Creating lookup paths for Amundsen...
2025-05-05 23:12:57,471 - __main__ - INFO - Adding tag usage counters...
2025-05-05 23:12:57,558 - __main__ - INFO - Verifying tag setup...
2025-05-05 23:12:57,707 - __main__ - INFO - Tag 'personal_info' is connected to 2 tables
2025-05-05 23:12:57,707 - __main__ - INFO - Tag 'experiment' is connected to 3 tables
2025-05-05 23:12:57,707 - __main__ - INFO - Tag 'user_data' is connected to 2 tables
2025-05-05 23:12:57,707 - __main__ - INFO - Tag 'PII' is connected to 1 tables
2025-05-05 23:12:57,707 - __main__ - INFO - Tag 'personal_data' is connected to 1 tables
2025-05-05 23:12:57,708 - __main__ - INFO - Tag 'user_info' is connected to 1 tables
2025-05-05 23:12:57,708 - __main__ - INFO - Tags fixed successfully
2025-05-05 23:12:57,708 - __main__ - INFO - Adding Amundsen-specific metadata...
2025-05-05 23:12:58,663 - __main__ - INFO - Amundsen-specific metadata added
2025-05-05 23:12:58,663 - __main__ - INFO - Verifying tag setup for Amundsen...
2025-05-05 23:12:58,804 - __main__ - INFO - Found table: user_data
2025-05-05 23:12:58,828 - __main__ - INFO - Found 6 tags
2025-05-05 23:12:58,877 - __main__ - INFO - Found 10 tag-to-table relationships
2025-05-05 23:12:58,877 - __main__ - INFO - Tag 'personal_info' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,877 - __main__ - INFO - Tag 'personal_info' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,878 - __main__ - INFO - Tag 'experiment' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,878 - __main__ - INFO - Tag 'experiment' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,878 - __main__ - INFO - Tag 'experiment' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,878 - __main__ - INFO - Tag 'user_data' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,879 - __main__ - INFO - Tag 'user_data' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,879 - __main__ - INFO - Tag 'PII' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,879 - __main__ - INFO - Tag 'personal_data' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,879 - __main__ - INFO - Tag 'user_info' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,911 - __main__ - INFO - Found 6 search result nodes for tags
2025-05-05 23:12:58,976 - __main__ - INFO - Found 1 tables with display names for Amundsen
2025-05-05 23:12:59,012 - __main__ - INFO - Found 6 tag dashboard connections
2025-05-05 23:12:59,060 - __main__ - INFO - Found 6 tag badge connections
2025-05-05 23:12:59,105 - __main__ - INFO - Found 10 TAGGED relationships from tags to tables
2025-05-05 23:12:59,158 - __main__ - INFO - Found 6 tags with proper Amundsen properties
```



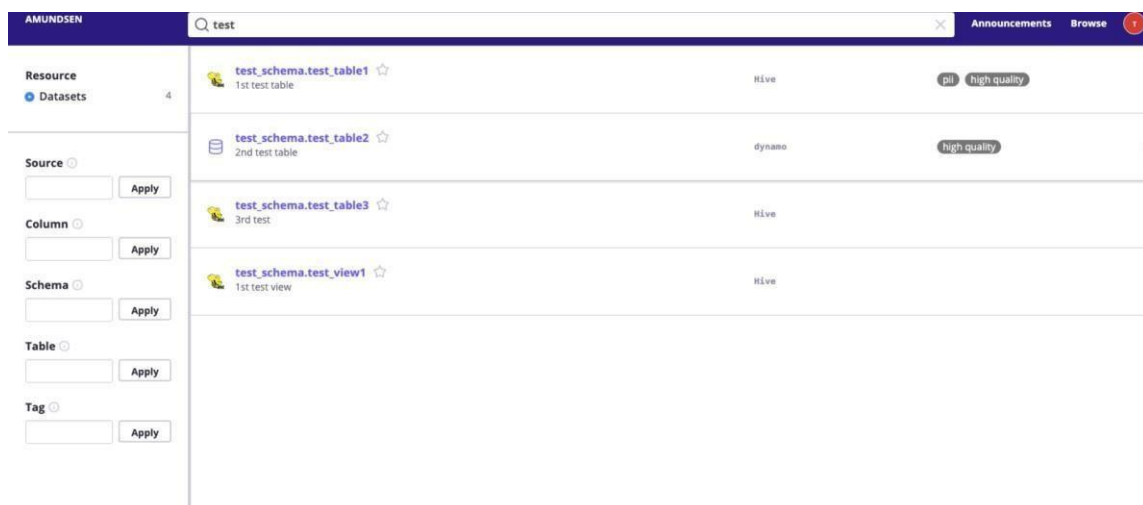
Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

localhost:5000

Metatags Personal_Info, User_Info, Experiment_Badge



DATABASES



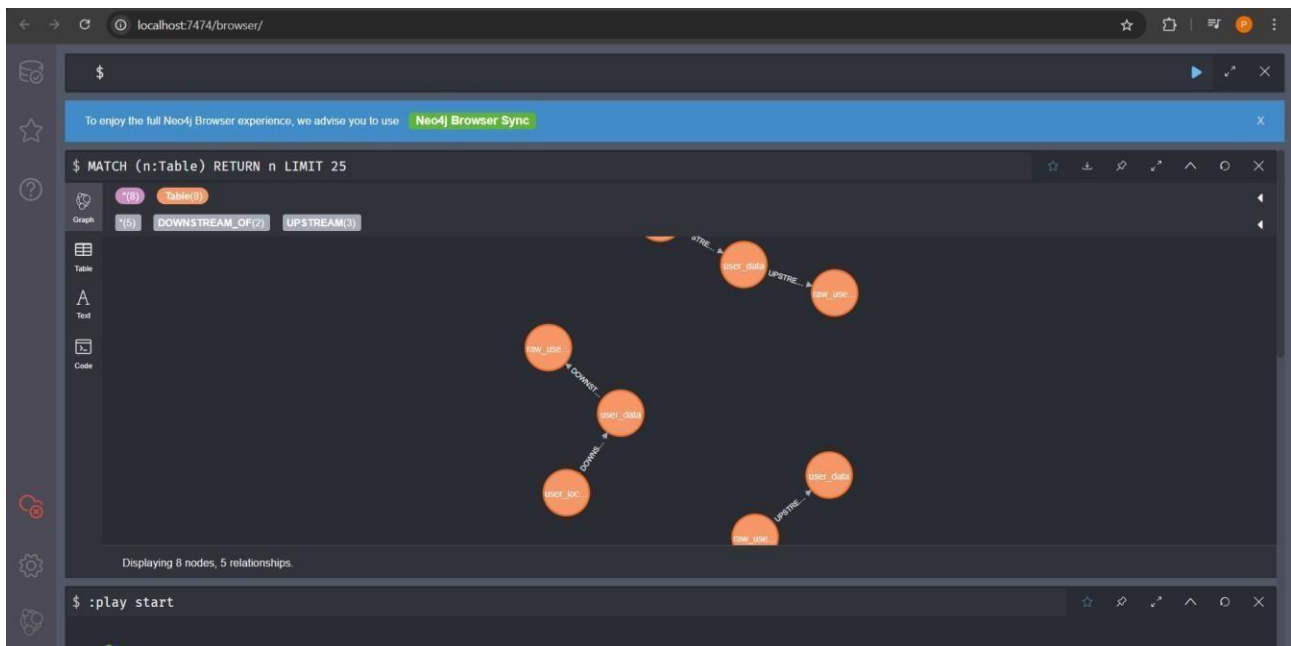


Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

DATASET Values

default.event_amundsenfrontend_user_action						
	USER_VALUE	POS_ARGS_JSON	START_EPOCH_MS	HTTP_REQUEST_ID	ERROR	PRODUCER_SERVICE
	dmittow@lyft.com	[]	1593536550891	635a2231-8c06-49e5-9e60-74b643f58a3e	null	amundsenfront
	alewis@lyft.com	[]	1593507028600	c848704-72d1-4c09-bb6a-c08a509000	null	amundsenfront
vive/default/event_pricing_data_worker_executed", "value": null]	jamiekang@lyft.com	[]	1593539861251	51c0364e-34ee-4c80-bb09-ae9c7c13e9e	null	amundsenfront
	johnchen@lyft.com	[]	1593539692208	01d58694-42e4-44b6-8fcc-56929cd46214	null	amundsenfront
	cindyzhang@lyft.com	[]	1593536621998	c7b394a-c1ea-4c62-be82-d0c3c8b5530	null	amundsenfront
	mingyangliu@lyft.com	[]	1593539619093	630cc144-bc2d-4960-a7cb-4fb2486709d	null	amundsenfront
"location": "search", "value": null]	johnchen@lyft.com	[]	1593538012325	3617e16b-bbe4-45e9-8e56-2d9ba0d3f34b	null	amundsenfront
/hive/core/fact_rides", "value": null]	plipkady@lyft.com	[]	1593537176335	1e80925d-6400-4bdc-84bd-139863d52596	null	amundsenfront
	ksineen@lyft.com	[]	1593538058370	3d7cde4-fa34-4524-82ac-f80fa26fa188	null	amundsenfront
	ksineen@lyft.com	[]	1593538038768	661e9bb7-9a19-49ae-8a70-ccdc319c2bb9	null	amundsenfront
	ddobson@lyft.com	[]	1593539378651	541b645b-1a5b-4c0c-a204-6e084c9f331	null	amundsenfront
	mingyangliu@lyft.com	[]	1593539609464	e142d6d9-a51c-4994-9164-27d92ac435d4	null	amundsenfront
	ckiszek@lyft.com	[]	1593539769808	abf697f1-644c-4d48-89d7-49drc81e5a0	null	amundsenfront
	ccchung@lyft.com	[]	1593537593829	58416d3-7d07-455b-a85e-609596b612bf	null	amundsenfront
	delahstane@lyft.com	[]	1593536573215	014100c5-929b-4ca1-8a9c-3da0f5734d086	null	amundsenfront
in": null, "bounced_code": null, "bounced_description": null, "dropped_reason": null, "dropped_code": null, "clicked_url": null]	mailer.mailer@lyft.com	[]	1593536425497	7853d247-a0e2-4091-b092-81a076c1afdd	null	amundsenfront
	cindyzhang@lyft.com	[]	1593536621999	b30a6798-843e-4543-a40c-e6bd8bc0be10	null	amundsenfront
	esagay@lyft.com	[]	1593537727233	e2046500-a477-4544-807b-ccc40e6d2e19	null	amundsenfront
	csun@lyft.com	[]	1593539926096	57a7a402-2657-4c0d-91c2-22755f704652	null	amundsenfront
	lukezhang@lyft.com	[]	1593537620277	5730a168-8395-4531-9db6-2b49a3978881	null	amundsenfront
	dsalman@lyft.com	[]	159353791060	915e998f-d933-4871-b79f-fe899d9418d2	null	amundsenfront

Neo4j Data lineage

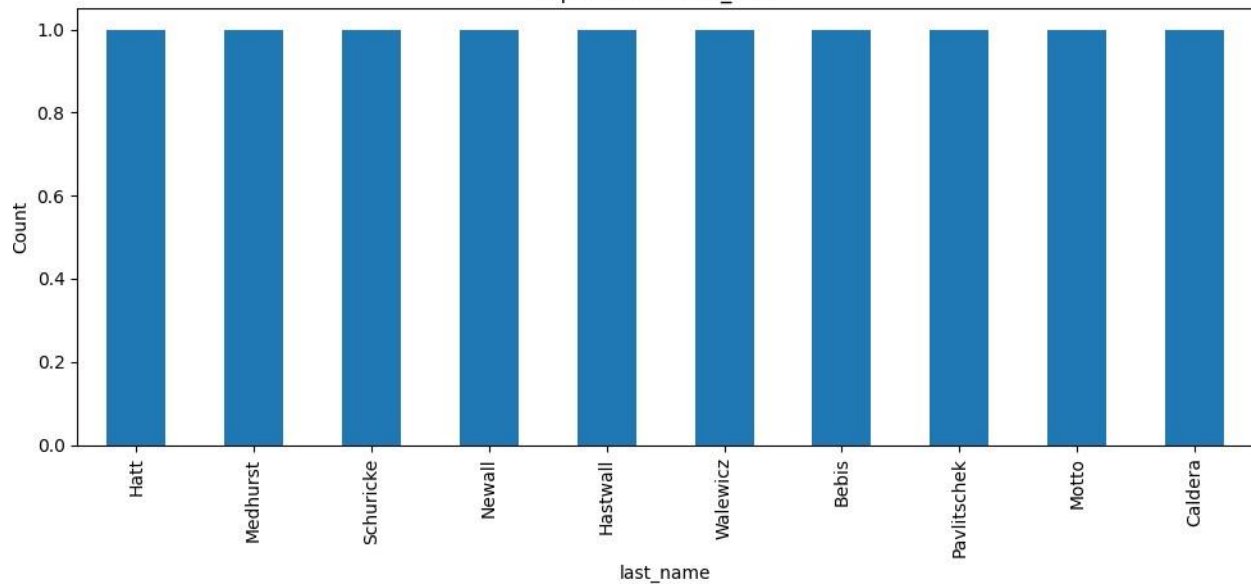


Lineage Tracking



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

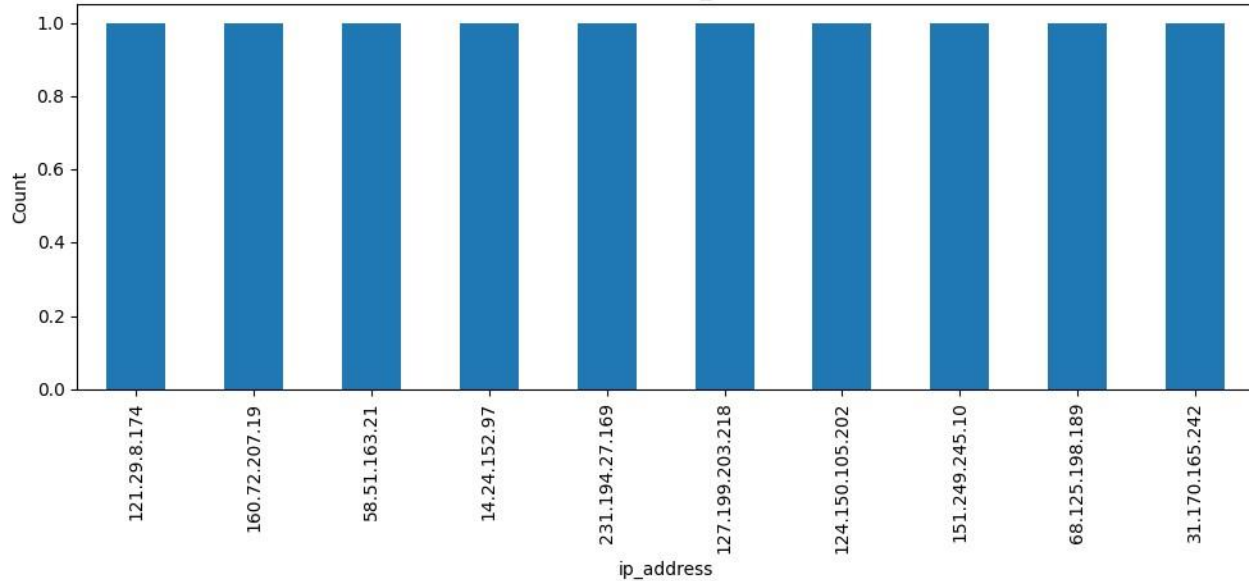
Top values in last_name



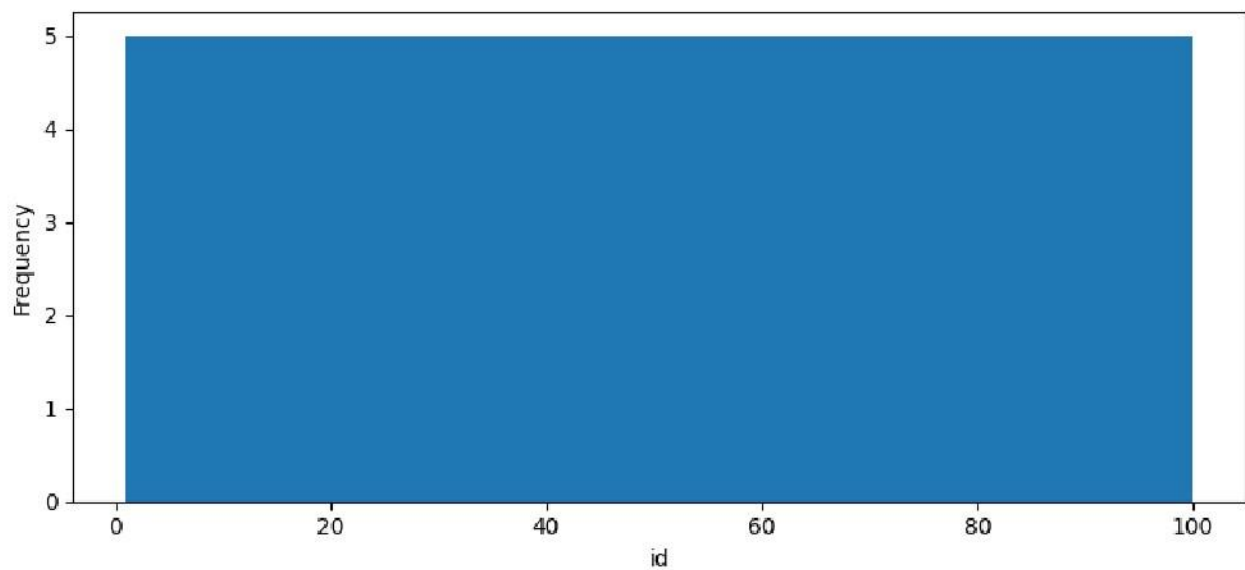


Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

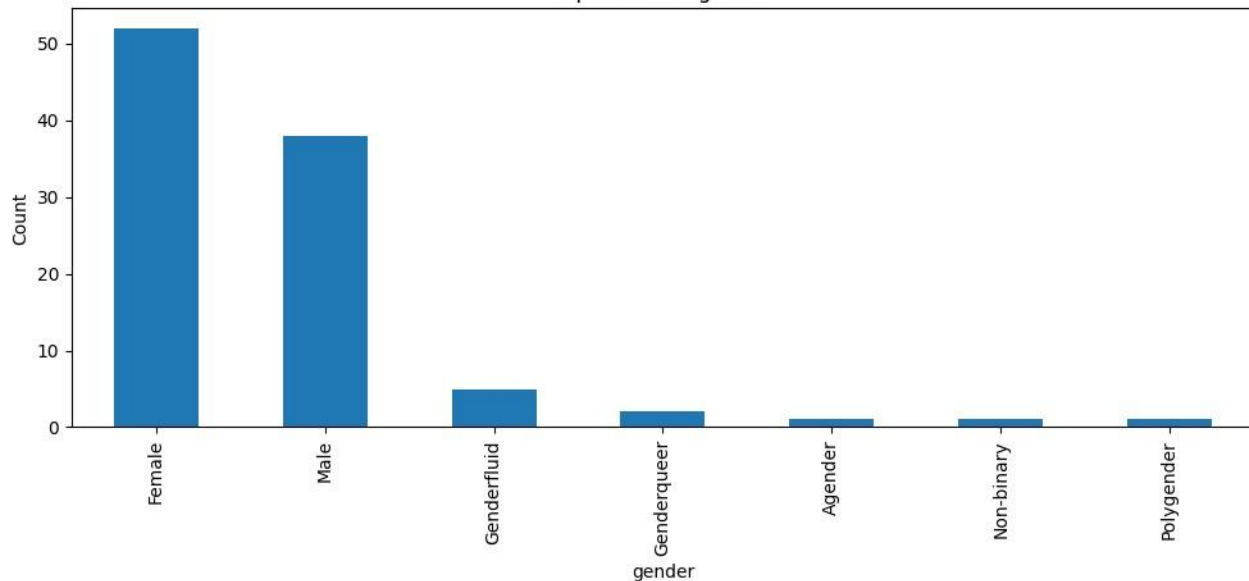
Top values in ip_address



Distribution of id

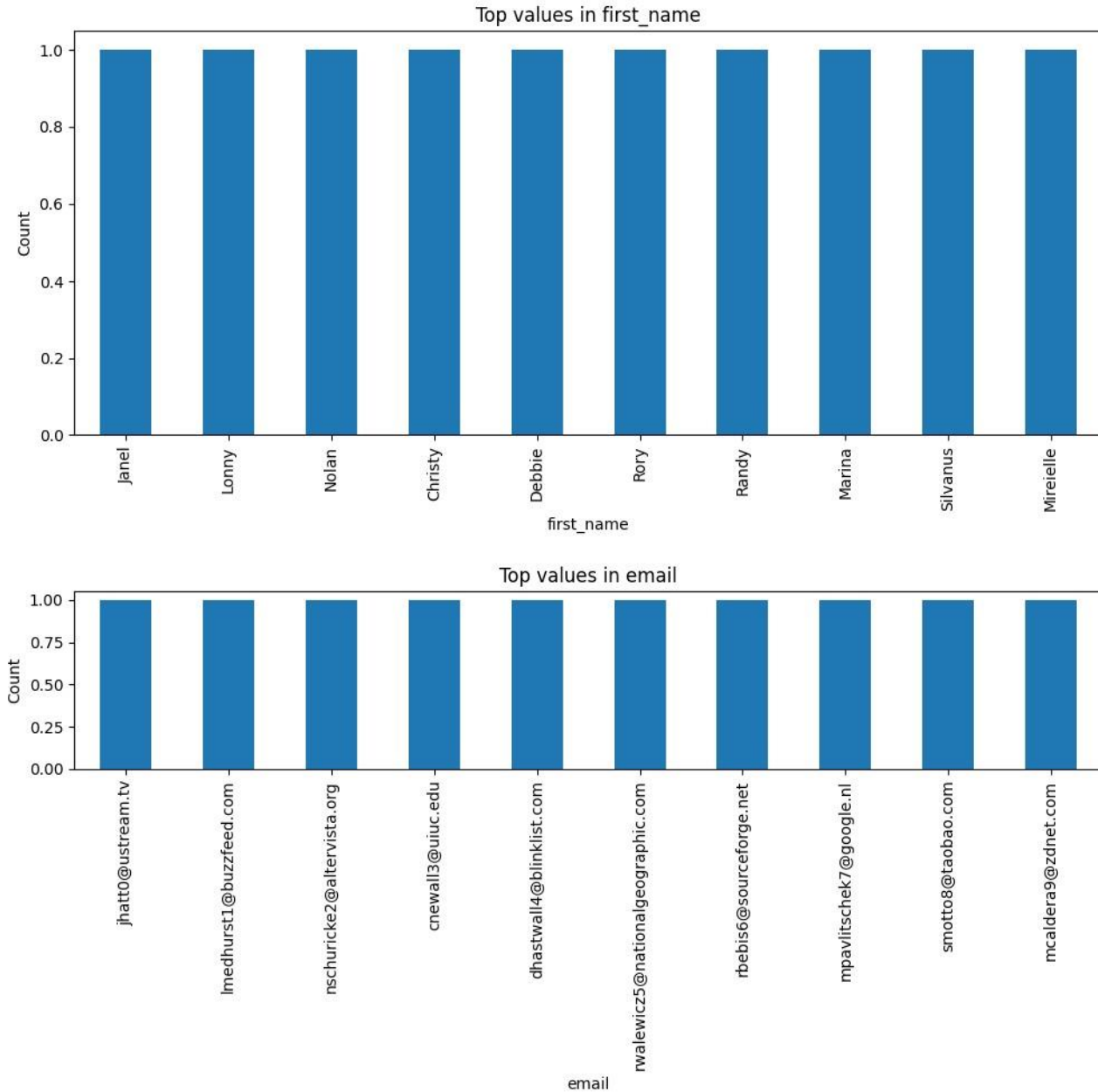


Top values in gender





Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)



Data Sharing Portal

```
brett@docker:~$ sudo docker container ls
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
782aa24b8447   ckan-docker-ckan  "/srv/app/start_ckan..." 3 minutes ago  Up 2 minutes (health: starting)  0.0.0.0:50
046d6c221765   ckan/ckan-solr:2.9-solr8  "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes (healthy)          8983/tcp
46f557482958   ckan/ckan-base-datapusher:0.0.19  "sh -c 'uwsgi --plug..." 3 minutes ago  Up 3 minutes (healthy)          8800/tcp
er
3a0935bc4dee   redis:6         "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes (healthy)          6379/tcp
c63f1b04f5c7   ckan-docker-db   "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes (healthy)          5432/tcp
brett@docker:~$
brett@docker:~$ sudo docker container ls
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
782aa24b8447   ckan-docker-ckan  "/srv/app/start_ckan..." 4 minutes ago  Up 2 minutes (health: starting)  0.0.0.0:50
046d6c221765   ckan/ckan-solr:2.9-solr8  "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes (healthy)          8983/tcp
46f557482958   ckan/ckan-base-datapusher:0.0.19  "sh -c 'uwsgi --plug..." 4 minutes ago  Up 4 minutes (healthy)          8800/tcp
er
3a0935bc4dee   redis:6         "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes (healthy)          6379/tcp
c63f1b04f5c7   ckan-docker-db   "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes (healthy)          5432/tcp
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

← → ↻ Not Secure | https://localhost:8443

news finance music books channels virus logins courses tech books running desks (Desk GW-... Lucid ckan-docker xloader

Datasets Organizations Groups About Se

Welcome to CKAN

This is a nice introductory paragraph about CKAN or the site in general. We don't have any copy to go here yet but soon we will



This is a featured section

Search data

Popular tags

[About CKAN](#)
[CKAN API](#)
[CKAN Association](#)
[OPEN DATA](#)

Powered by 
Language: English

← → X Not Secure | https://localhost:8443/user/ckan_admin

news finance music books channels virus logins courses tech books running desks (Desk GW-... Lucid ckan-dock

Datasets Organizations Groups A

🏠 / Users / ckan_admin



ckan_admin

You have not provided a biography.

Followers	Datasets
0	0

Username
ckan_admin

Email Private
your_email@example.com

Member Since

[Datasets](#) [API Tokens](#)

You haven't created any datasets. [Create one now?](#)

https://localhost:8443/dataset/

CKAN 2.10 Pac....mp4 CKAN 2.10 Sour....mp4

Conclusion: In conclusion, the implementation showed data cataloging and sharing services using open-source tools like Amundsen and CKAN. The setup enabled automated metadata ingestion, governance, and secure dataset sharing, demonstrating practical applications of modern data management frameworks



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

T.Y B. Tech, Semester: VI

Academic Year 2024-25

Name: Gargi Sukhatankar

Batch: B1

SAPID: 60019230125

Roll No: B027

Experiment No. 10

Aim: To Create Pipeline for the Gold Layer by using ETL Tools (dbt)

Theory:

=>Medallion Architecture: A Detailed Overview

The medallion architecture is a data design pattern widely used in modern data engineering to organize and process data through progressive layers of refinement. Each layer represents a different stage of data processing, named after precious metals to signify increasing value.

=>Core Layers of the Medallion Architecture

1. Bronze Layer (Raw Layer)

- **Purpose:** Acts as the landing zone for raw data
- **Characteristics:**
 - Contains unmodified source data in its original format
 - Preserves the complete source system information
 - Often includes metadata like ingestion timestamps and source identifiers
 - No business logic or transformations applied
 - Serves as a historical archive of raw data
- **Implementation:** Usually stored as tables with minimal schema enforcement

2. Silver Layer (Cleansed Layer)

- **Purpose:** Standardizes and cleanses data
- **Characteristics:**
 - Contains validated and cleansed data
 - Enforces basic data quality rules
 - Removes duplicates and standardizes formats
 - Applies type conversions and basic normalization
 - Resolves schema inconsistencies
 - May include initial derived fields
- **Implementation:** Often implemented as views or tables with standardized schemas

3. Gold Layer (Business Layer)



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

- **Purpose:** Provides business-ready data products
- **Characteristics:**
 - Contains aggregated, transformed data optimized for consumption
 - Implements complex business logic and calculations
 - Organizes data into dimensional models, fact tables, or specific domain datasets
 - Optimized for query performance and analytics
 - Ready for direct consumption by BI tools, dashboards, and applications
 - Contains business metrics, KPIs, and domain-specific aggregates
- **Implementation:** Usually materialized as tables optimized for read performance

=>Benefits of the Medallion Architecture

1. **Separation of Concerns:** Each layer has a specific purpose, making the system easier to maintain
2. **Data Lineage:** Clear tracking of how data moves and transforms through the pipeline
3. **Incremental Processing:** Enables efficient processing of only new or changed data
4. **Recovery and Debugging:** Ability to reprocess data from any layer
5. **Isolation of Complexity:** Complex transformations are built progressively
6. **Reusability:** Intermediate layers can be reused for multiple downstream purposes
7. **Quality Control:** Progressive validation and quality checks at each layer

dbt (Data Build Tool)

dbt is an open-source transformation tool that enables data analysts and engineers to transform data in their warehouse using SQL-based transformations.

Key Components of dbt

1. **Models:** SQL files that transform source data into final outputs
2. **Sources:** Configurations that define the external tables dbt will query
3. **Tests:** Assertions about your data to ensure quality and correctness
4. **Documentation:** Automated documentation generation for your data models
5. **Macros:** Reusable SQL snippets (similar to functions)



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

6. **Seeds:** CSV files that can be loaded as tables
7. **Snapshots:** Point-in-time snapshots of changing data
8. **Materializations:** How models are physically implemented (table, view, etc.)

dbt Architecture and Implementation Implementation Process

1. Project Setup:

- Initialize a dbt project (dbt init)
- Configure connection profiles (profiles.yml)
- Define project settings (dbt_project.yml)

2. Model Definition

- Create SQL files for transformations
- Organize models in folder structure
- Define dependencies between models

3. Testing and Documentation:

- Add data tests
- Document models and columns

Generate documentation ○

4. Execution:

- Run models (dbt run)

Test data (dbt test)

- Generate documentation (dbt docs generate)

dbt Pipeline Flow

1. **Compilation:** dbt compiles SQL models into executable SQL
2. **Dependency Resolution:** dbt builds a directed acyclic graph (DAG) of dependencies



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

3. **Execution:** Models are executed in dependency order
4. **Materialization:** Results are materialized according to defined strategies
5. **Testing:** Tests are run against the materialized models
6. **Documentation:** Docs are generated based on model definitions and metadata **dbt**

Materializations

1. **Table:** Full rebuild of table each run
2. **View:** Creates a SQL view (no data storage)
3. **Incremental:** Processes only new/changed records
4. **Ephemeral:** Not materialized but compiled into dependent models

Implementation of Medallion Architecture with dbt

Project Structure

```
models/
bronze/
|
|— sources.yml      # External data source definitions
|— raw_orders.sql   # Bronze layer raw data models
|— raw_customers.sql
|— silver/
|— clean_orders.sql # Silver layer standardized models
|— clean_customers.sql
|— gold/
|— customer_metrics.sql # Gold layer business models
|— daily_sales.sql
```

Bronze Layer Implementation

The bronze layer in dbt typically connects to source systems and brings in raw data with minimal transformation:

```
sql
-- bronze/raw_orders.sql
{{ config(materialized='table') }}
```

```
SELECT
  *,
  CURRENT_TIMESTAMP() as ingestion_timestamp,
  'source_system' as data_source
FROM {{ source('operational_db', 'orders') }}
```

Silver Layer Implementation

The silver layer applies data cleansing and standardization:

```
sql
-- silver/clean_orders.sql
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
{{ config(materialized='view') }}
```

```
SELECT
  order_id,
  customer_id,
  TRIM(product) as product,
  COALESCE(amount, 0) as amount,
  STR_TO_DATE(order_date, '%Y-%m-%d') as order_date
FROM {{ ref('raw_orders') }}
WHERE order_id IS NOT NULL
```

Gold Layer Implementation

The gold layer creates business-ready models with metrics and calculations:

sql

```
-- gold/customer_metrics.sql
{{ config(materialized='table') }}
```

```
WITH customer_orders AS (
  SELECT
    c.customer_id,
    c.customer_name,
    COUNT(o.order_id) AS total_orders,
    SUM(o.amount) AS total_spent,
    MAX(o.order_date) AS last_order_date
  FROM {{ ref('clean_customers') }} c
  LEFT JOIN {{ ref('clean_orders') }} o ON c.customer_id = o.customer_id
  GROUP BY c.customer_id, c.customer_name
)
```

```
SELECT
  customer_id,
  customer_name,
  COALESCE(total_orders, 0) AS total_orders,
  COALESCE(total_spent, 0) AS total_spent,
  last_order_date,
  CASE
    WHEN total_spent > 300 THEN 'High Value'
    WHEN total_spent > 100 THEN 'Medium Value'
    ELSE 'Low Value'
  END AS customer_segment
FROM customer_orders
```

Implementation of Data Testing dbt

enables data quality testing at each layer:

yaml

```
# models/gold/schema.yml
```

```
version: 2
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

models:

- name: customer_metrics description: "Customer metrics with segmentation"
columns:
- name: customer_id description: "Unique customer identifier" tests:
- unique
- not_null
- name: total_spent description: "Total amount spent by customer" tests:
- not_null

dbt Pipeline Flow in the Medallion

Architecture

1. **Source Connection:** dbt connects to source systems defined in sources.yml
2. **Bronze Layer Creation:** Raw data is loaded into bronze layer models
3. **Silver Layer Transformation:** Bronze data is cleansed and standardized
4. **Gold Layer Aggregation:** Silver data is transformed into business metrics
5. **Testing:** Tests verify data quality at each layer
6. **Documentation:** The entire data pipeline is documented

Benefits of Using dbt for Medallion Architecture

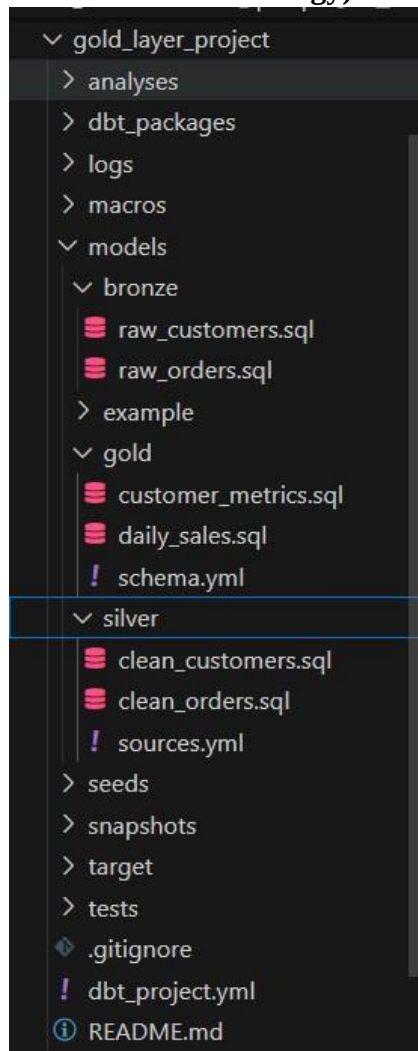
1. **Modularity:** Each transformation step is defined in separate SQL files
2. **Version Control:** All transformations are version-controlled
3. **Testing:** Built-in testing ensures data quality
4. **Documentation:** Automated documentation of the entire data pipeline
5. **Dependency Management:** Automatic handling of model dependencies
6. **Incremental Processing:** Efficient processing of new/changed data
7. **Reusability:** Common logic can be extracted into macros
8. **Observability:** Clear visualization of the entire data pipeline

Output:

- dbt init gold_layer_project



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)



- **dbt debug**

```
PS E:\Migration\Temp\DE_expt\gold_layer_project> dbt debug
19:12:15 Running with dbt=1.7.19
19:12:15 dbt version: 1.7.19
19:12:15 python version: 3.13.1
19:12:15 python path: E:\Migration\Temp\DE_expt\amundsen\databuilder\example\scripts\myenv\Scripts\python.exe
19:12:15 os info: Windows-11-10.0.26100-SP0
19:12:15 Using profiles dir at C:\Users\kalpe\.dbt
19:12:15 Using profiles.yml file at C:\Users\kalpe\.dbt\profiles.yml
19:12:15 Using dbt_project.yml file at E:\Migration\Temp\DE_expt\gold_layer_project\dbt_project.yml
19:12:15 adapter type: mysql
19:12:15 adapter version: 1.7.0
19:12:15 Configuration:
19:12:15   profiles.yml file [OK found and valid]
19:12:15   dbt_project.yml file [OK found and valid]
19:12:15 Required dependencies:
19:12:15   - git [OK found]

19:12:15 Connection:
19:12:15   server: 127.0.0.1
19:12:15   unix_socket: None
19:12:15   port: 3306
19:12:15   database: None
19:12:15   schema: gold_layer_db
19:12:15   user: root
19:12:15 Registered adapter: mysql=1.7.0
19:12:15 Connection test: [OK connection ok]
```

- **dbt run --select bronze**



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
PS E:\Migration\Temp\DE_expt\gold_layer_project> dbt run --select bronze
19:12:39 Running with dbt=1.7.19
19:12:40 Registered adapter: mysql=1.7.0
19:12:40 Unable to do partial parsing because saved manifest not found. Starting full parse.
19:12:41 Found 8 models, 9 tests, 2 sources, 0 exposures, 0 metrics, 375 macros, 0 groups, 0 semantic models
19:12:41
19:12:41 Concurrency: 1 threads (target='dev')
19:12:41
19:12:41 1 of 2 START sql table model gold_layer_db.raw_customers ..... [RUN]
19:12:42 1 of 2 OK created sql table model gold_layer_db.raw_customers ..... [SUCCESS 3 in 0.28s]
19:12:42 2 of 2 START sql table model gold_layer_db.raw_orders ..... [RUN]
19:12:42 2 of 2 OK created sql table model gold_layer_db.raw_orders ..... [SUCCESS 5 in 0.12s]
19:12:42
19:12:42 Finished running 2 table models in 0 hours 0 minutes and 0.60 seconds (0.60s).
19:12:42 Completed successfully
19:12:42
19:12:42 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 TOTAL=2
```

- **dbt run --select silver**

```
PS E:\Migration\Temp\DE_expt\gold_layer_project> dbt run --select silver
19:12:53 Running with dbt=1.7.19
19:12:53 Registered adapter: mysql=1.7.0
19:12:53 Found 8 models, 9 tests, 2 sources, 0 exposures, 0 metrics, 375 macros, 0 groups, 0 semantic models
19:12:53
19:12:53 Concurrency: 1 threads (target='dev')
19:12:53
19:12:53 1 of 2 START sql view model gold_layer_db.clean_customers ..... [RUN]
19:12:54 1 of 2 OK created sql view model gold_layer_db.clean_customers ..... [SUCCESS 0 in 0.23s]
19:12:54 2 of 2 START sql view model gold_layer_db.clean_orders ..... [RUN]
19:12:54 2 of 2 OK created sql view model gold_layer_db.clean_orders ..... [SUCCESS 0 in 0.10s]
19:12:54
19:12:54 Finished running 2 view models in 0 hours 0 minutes and 0.51 seconds (0.51s).
19:12:54 Completed successfully
19:12:54
19:12:54 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 TOTAL=2
```

- **dbt run --select gold**

```
PS E:\Migration\Temp\DE_expt\gold_layer_project> dbt run --select gold
19:13:02 Running with dbt=1.7.19
19:13:03 Registered adapter: mysql=1.7.0
19:13:03 Found 8 models, 9 tests, 2 sources, 0 exposures, 0 metrics, 375 macros, 0 groups, 0 semantic models
19:13:03
19:13:03 Concurrency: 1 threads (target='dev')
19:13:03
19:13:03 1 of 2 START sql table model gold_layer_db.customer_metrics ..... [RUN]
19:13:03 1 of 2 OK created sql table model gold_layer_db.customer_metrics ..... [SUCCESS 3 in 0.20s]
19:13:03 2 of 2 START sql table model gold_layer_db.daily_sales ..... [RUN]
19:13:03 2 of 2 OK created sql table model gold_layer_db.daily_sales ..... [SUCCESS 3 in 0.09s]
19:13:03
19:13:03 Finished running 2 table models in 0 hours 0 minutes and 0.43 seconds (0.43s).
19:13:03 Completed successfully
19:13:03
19:13:03 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 TOTAL=2
```

- **dbt run**



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
PS E:\Migration\Temp\DE_expt\gold_layer_project> dbt run
19:13:13 Running with dbt=1.7.19
19:13:13 Registered adapter: mysql=1.7.0
19:13:13 Found 8 models, 9 tests, 2 sources, 0 exposures, 0 metrics, 375 macros, 0 groups, 0 semantic models
19:13:13
19:13:13 Concurrency: 1 threads (target='dev')
19:13:13
19:13:13 1 of 8 START sql table model gold_layer_db.my_first_dbt_model ..... [RUN]
19:13:14 1 of 8 OK created sql table model gold_layer_db.my_first_dbt_model ..... [SUCCESS 2 in 0.30s]
19:13:14 2 of 8 START sql table model gold_layer_db.raw_customers ..... [RUN]
19:13:14 2 of 8 OK created sql table model gold_layer_db.raw_customers ..... [SUCCESS 3 in 0.18s]
19:13:14 3 of 8 START sql table model gold_layer_db.raw_orders ..... [RUN]
19:13:14 3 of 8 OK created sql table model gold_layer_db.raw_orders ..... [SUCCESS 5 in 0.18s]
19:13:14 4 of 8 START sql view model gold_layer_db.my_second_dbt_model ..... [RUN]
19:13:14 4 of 8 OK created sql view model gold_layer_db.my_second_dbt_model ..... [SUCCESS 0 in 0.11s]
19:13:14 5 of 8 START sql view model gold_layer_db.clean_customers ..... [RUN]
19:13:14 5 of 8 OK created sql view model gold_layer_db.clean_customers ..... [SUCCESS 0 in 0.12s]
19:13:14 6 of 8 START sql view model gold_layer_db.clean_orders ..... [RUN]
19:13:14 6 of 8 OK created sql view model gold_layer_db.clean_orders ..... [SUCCESS 0 in 0.12s]
19:13:14 7 of 8 START sql table model gold_layer_db.customer_metrics ..... [RUN]
19:13:14 7 of 8 OK created sql table model gold_layer_db.customer_metrics ..... [SUCCESS 3 in 0.21s]
19:13:14 8 of 8 START sql table model gold_layer_db.daily_sales ..... [RUN]
19:13:15 8 of 8 OK created sql table model gold_layer_db.daily_sales ..... [SUCCESS 3 in 0.20s]
19:13:15
19:13:15 Finished running 5 table models, 3 view models in 0 hours 0 minutes and 1.65 seconds (1.65s).
19:13:15
19:13:15 Completed successfully
19:13:15
19:13:15 Done. PASS=8 WARN=0 ERROR=0 SKIP=0 TOTAL=8
```

• dbt docs generate

```
PS E:\Migration\Temp\DE_expt\gold_layer_project> dbt docs generate
19:13:24 Running with dbt=1.7.19
19:13:25 Registered adapter: mysql=1.7.0
19:13:25 Found 8 models, 9 tests, 2 sources, 0 exposures, 0 metrics, 375 macros, 0 groups, 0 semantic models
19:13:25
19:13:25 Concurrency: 1 threads (target='dev')
19:13:25
19:13:25 Building catalog
19:13:25 Catalog written to E:\Migration\Temp\DE_expt\gold_layer_project\target\catalog.json
PS E:\Migration\Temp\DE_expt\gold_layer_project> dbt docs serve
19:13:34 Running with dbt=1.7.19
```

• dbt docs serve

```
PS E:\Migration\Temp\DE_expt\gold_layer_project> dbt docs serve
19:31:02 Running with dbt=1.7.19
Serving docs at 8080
To access from your browser, navigate to: http://localhost:8080

Press Ctrl+C to exit.
127.0.0.1 - - [06/May/2025 01:01:03] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [06/May/2025 01:01:03] "GET /manifest.json?cb=1746473463724 HTTP/1.1" 200 -
127.0.0.1 - - [06/May/2025 01:01:03] "GET /catalog.json?cb=1746473463724 HTTP/1.1" 200 -
127.0.0.1 - - [06/May/2025 01:01:04] code 404, message File not found
127.0.0.1 - - [06/May/2025 01:01:04] "GET /%7Brequire('./assets/favicons/favicon.ico')%7D HTTP/1.1" 404 -
127.0.0.1 - - [06/May/2025 01:01:05] code 404, message File not found
127.0.0.1 - - [06/May/2025 01:01:05] "GET /index.php/apps/files/preview-service-worker.js HTTP/1.1" 404 -
```

OUTPUT:

dbt tool:



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

The screenshot shows the dbt interface with the 'clean_orders' view selected. The left sidebar shows the project structure with 'gold_layer_project' and its sub-projects 'models', 'gold', and 'silver'. The 'clean_orders' view is highlighted under the 'silver' project. The main panel displays the 'clean_orders' view details, including its description, columns, and constraints.

clean_orders view

Details Description Columns Referenced By Depends On Code

Details

TAGS	OWNER	TYPE	PACKAGE	LANGUAGE	RELATION	ACCESS	VERSION	CONTRACT
untagged		view	gold_layer_project	sql	gold_layer_db.clean_orders	protected		Not Enforced

Description

This model is not currently documented

Columns

COLUMN	TYPE	DESCRIPTION	CONSTRAINTS	TESTS	MORE
order_id	bigint				
customer_id	bigint				
product	varchar(9)				
amount	decimal(5,2)				

MYSQL Workbench:

The screenshot shows the MySQL Workbench interface with a query executed. The query is a SELECT statement that joins data from 'raw_orders', 'raw_customers', 'clean_orders', 'clean_customers', 'customer_metrics', and 'daily_sales'. The results are displayed in a table with columns: sale_date, orders_count, daily_revenue, unique_customers, and average_order_value. The bottom panel shows the output of the query, including the time taken for each row to be returned.

Query 1

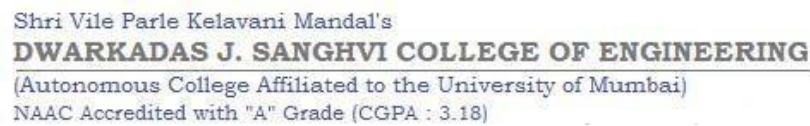
```
SELECT * FROM raw_customers;  
-- Check silver layer views  
SELECT * FROM clean_orders;  
SELECT * FROM clean_customers;  
-- Check gold layer tables  
SELECT * FROM customer_metrics;  
SELECT * FROM daily_sales;
```

Result Grid

sale_date	orders_count	daily_revenue	unique_customers	average_order_value
2024-05-01	2	350.00	2	175.000000
2024-05-02	1	75.50	1	75.500000
2024-05-03	2	500.00	2	250.000000

Output

#	Time	Action	Message	Duration / Fetch
11	00:47:57	SELECT * FROM raw_orders LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
12	00:47:57	SELECT * FROM raw_customers LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
13	00:47:57	SELECT * FROM clean_orders LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
14	00:47:57	SELECT * FROM clean_customers LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
15	00:47:57	SELECT * FROM customer_metrics LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec
16	00:47:57	SELECT * FROM daily_sales LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.000 sec



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

Bronze layer:

raw_orders

	order_id	customer_id	product	amount	order_date
▶	1	101	Product A	200.00	2024-05-01
	2	102	Product B	150.00	2024-05-01
	3	101	Product C	75.50	2024-05-02
	4	103	Product A	200.00	2024-05-03
	5	102	Product D	300.00	2024-05-03

raw_customers

	customer_id	customer_name	email	country
▶	101	John Doe	john@example.com	USA
	102	Jane Smith	jane@example.com	Canada
	103	Bob Johnson	bob@example.com	USA

Silver Layer:

clean_orders

	order_id	customer_id	product	amount	order_date
▶	1	101	Product A	200.00	2024-05-01
	2	102	Product B	150.00	2024-05-01
	3	101	Product C	75.50	2024-05-02
	4	103	Product A	200.00	2024-05-03
	5	102	Product D	300.00	2024-05-03

clean_customers

	order_id	customer_id	product	amount	order_date
▶	1	101	Product A	200.00	2024-05-01
	2	102	Product B	150.00	2024-05-01
	3	101	Product C	75.50	2024-05-02
	4	103	Product A	200.00	2024-05-03
	5	102	Product D	300.00	2024-05-03

Gold Layer:

customer_metrics

customer_metrics								
	customer_id	customer_name	email	country	total_orders	total_spent	last_order_date	customer_segment
▶	101	John Doe	john@example.com	USA	2	275.50	2024-05-02	Medium Value
	102	Jane Smith	jane@example.com	Canada	2	450.00	2024-05-03	High Value
	103	Bob Johnson	bob@example.com	USA	1	200.00	2024-05-03	Medium Value



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

daily_sales

	sale_date	orders_count	daily_revenue	unique_customers	average_order_value
▶	2024-05-01	2	350.00	2	175.000000
	2024-05-02	1	75.50	1	75.500000
	2024-05-03	2	500.00	2	250.000000

Conclusion: In conclusion, the medallion architecture implemented with dbt creates a well-structured, maintainable data pipeline that progressively refines data from raw formats to business-ready insights. This approach addresses common data engineering challenges by separating concerns, maintaining data lineage, ensuring quality, and optimizing for both development efficiency and analytical performance.

Name: Gaigi P. Sukhatankar
Roll No: B027

SAPID: 60019220125
Branch: CSE(CB)

DE- ASSIGNMENT 1

Q-1. Explain in detail types of data Architecture - data warehouse, DWH characteristic, data lake and lambda architecture

⇒ (i) Data Warehouse

- Data warehouse (DWH) was proposed as a new type of database management system which would keep no transactional data but only summarized historical information for decision making purposes.
- DW's are modelled & structured differently, use different techniques for storage and retrieval and cater to a different set of users.
- Data warehouses are databases that store and maintain analytical data separately from transaction-oriented databases for the purpose of decision support.

Characteristics of DWH:

- Multidimensional conceptual view.
- Unlimited dimensions and aggregation levels.
- Unrestricted cross-dimensional operations.
- Dynamic sparse matrix handling.
- Client-server architecture.
- Multiuser support.
- Accessibility
- Transparency
- Intuitive data manipulation
- Inductive & deductive analysis.
- Flexible distributed reporting.

② Data Lake

A data lake is a centralized repository that allows you to store all your structured and unstructured data at any scale.

Primary components of data lake.

1) Metadata

→ Accountable for managing data structure.

→ can be used to boost computing efficiency & improve the usability.

2) Compute

→ Accountable for doing data crunching.

→ Computing engine can be combined with storage, as in a warehouse, or they might be entirely focused on computing.

3) Format

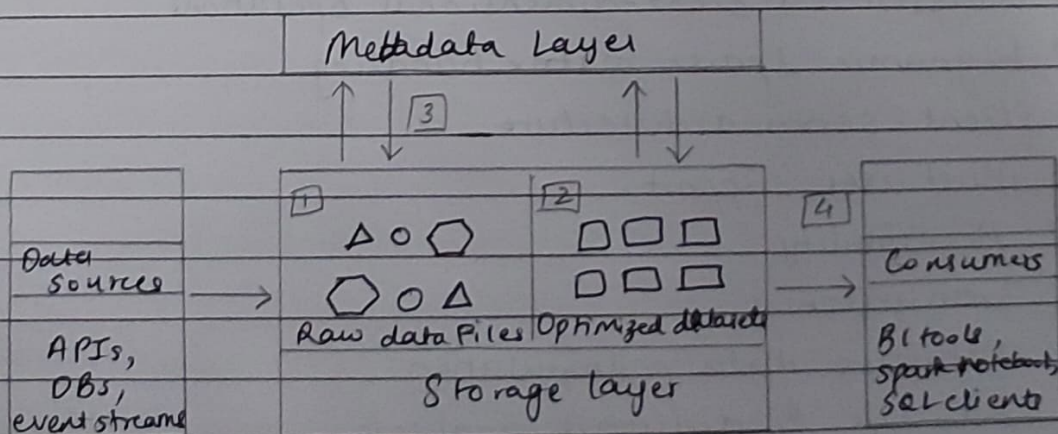
→ Data of any shape or format is stored at the storage layer.

→ Format has an impact on the available operations and the compatibility of compute engines.

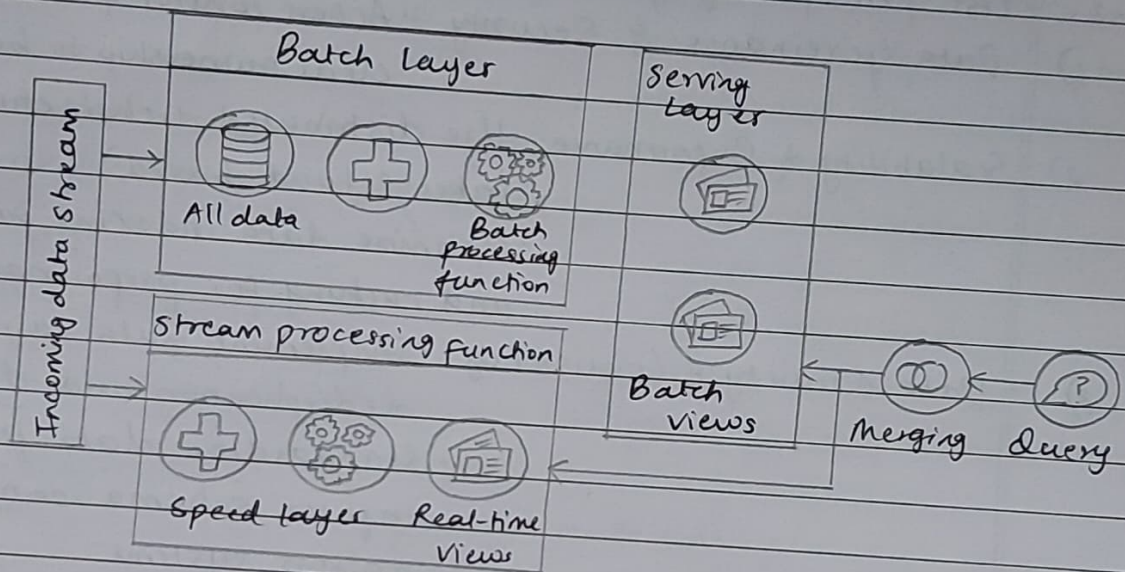
4) Storage

→ Where data is physically stored.

→ Durability, scalability and compatibility with other tools, engines and libraries are all requirements.



③ Lambda Architecture



Key Components

- **Batch Layer:** Saves incoming data as batch views.
- **Speed Layer:** Delivers data into batch layer.
- **Serving Layer:** Batch view and speed layer outcome traverse to serving layer.
- Lambda architecture provides a common architectural pattern for discussion.
- Provides a more clear picture of the complexities typically found in most organizations.
- **Challenges in tackling lambda architecture**
- Complete lambda architecture requires more than just a single system.
- Some challenges in delivering results to the business.
- Takes a lot of knowledge/expertise/technology to tackle.
- Not typically a first step in Big Data implementation.

Q.2. List principles of Good Data Architecture.

- 1) Data Governance & Security : Access control & encryption,
• clear ownership to be established
- 2) Scalability & Performance : • Use distributed architecture like data lakes (cloud-based) & warehouses
• Optimize data indexing, partitioning and caching for performance.
- 3) Data Quality & Consistency : • Implement data validation, deduplication and cleaning.
• Standardize data forms & enforce schema consistency across systems.
• Use real-time monitoring to detect and resolve data quality issues.
- 4) Interoperability & Integration : • Use APIs, ETL, pipelines and data streaming
• Adopt open standards (JSON, Avro, Parquet) for data exchange.
• Enable cross-platform compatibility for multi-cloud and hybrid environments.
- 5) Flexibility and Modularity : • Design data models that support agility and future modifications.
• Use microservices and event-driven architectures to decouple dependencies.
• Allow easy schema evolution without breaking existing applications.

- 6) Real Time & Batch Processing:
 • Implement both ETL and ELT.
 • Use tools like Apache Kafka, Spark and Flink for real-time data ingestion.
- 7) Metadata Management & Lineage:
 • Maintain metadata catalogs.
 • Track data lineage.
 • Use automated tools.
- 8) Cost Optimization:
 • Optimize storage costs using tiered storage strategies.
 • Implement auto scaling.
 • Regularly audit data usage & limit it.

Q-3. Explain different steps of Extraction, Transformation & Loading.

- ⇒ ① Determine all the target data needed in the data warehouse: Define scope of data.
- ② Determine all the data sources, both internal and external.
- ③ Prepare data mapping for Target Elements from sources: Map the fields from source systems to corresponding fields in the target schema.
- ④ Establish comprehensive data extraction rules: Define the rules and mechanisms for extracting data from various sources. Includes specifying filters, frequency (eg batch/real-time) and any conditional logic necessary during extraction.
- ⑤ Determine data transformation and cleansing rules: Specify how raw data will be standardized, cleansed and transformed to meet quality and format requirements. Includes deduplication, handling nulls, converting datatypes and applying business rules.

- ⑥ Plan for aggregate tables: Design and plan pre-aggregated data set to support performance optimized reporting. These tables provide summarized insights and improve efficiency in analytical tools.
- ⑦ Organize data staging area and test tools: Set up a staging environment where raw data is initially loaded for validation, transformation and testing. Ensure proper version control, logging and test automation are in place to maintain data integrity.
- ⑧ Write procedures for all data loads: Develop ETL Scripts or procedures to automate data loading process. Include logging, exception handling and rollback mechanisms to make pipeline reliable, repeatable and auditable.
- ⑨ ETL for dimension tables: Load and transform data for dimension tables first as these are used to provide context (eg customer, product, region) to the measurable data in fact tables.
- ⑩ ETL for fact tables: Finally, load the fact tables containing transactional or measurable data (eg sales, revenue, clicks). These tables should reference the previously loaded dimensions and are the foundation for business metrics and KPIs.

Q-4. Apply data techniques and processes in data loading environment

- ⇒ ① Batch vs Real-Time Loading: Loads large volume of data at scheduled intervals, ideal for systems where near real-time access isn't critical.
- ② Real-time: Enables continuous data flow, suitable for dynamic, fast-moving datasets like IoT, social media or financial feeds.

② Incremental Loading:

- Only new updated records are loaded rather than full dataset.
- Minimize data redundancy, reduces system load and boosts performance.
- Techniques: Change Data Capture (CDC), Timestamp based filtering or using delta files.

③ Data Validation & Integrity checks:

- Ensure referential integrity.
- Verify data types, null constraints and unique values.
- Log discrepancies for debugging or reruns.

④ Parallel Processing:

- Loads using multi-threading or distributed frameworks like Apache Spark or Talend to handle large-scale datasets efficiently.

⑤ Error handling and Recovery:

- Implement error logging mechanisms to track failed inserts or data mismatches.

⑥ Partitioning & Indexing

- Partition large tables to improve query and load performance.
- Apply proper indexing after data load to speed up analytical queries without slowing down the loading process.

⑦ Load Sequencing

- Dimension tables must be loaded before fact tables to ensure referential integrity.

⑧ Data Archival

- Archive old or unused data to reduce warehouse size and increase performance.

⑨ Automation & Scheduling:

- Use ETL orchestration tools (e.g. Apache Airflow, Azure Data Factory)

to automate loading process

② Monitoring and Auditing:

- Monitor data pipeline health and performance.
- Maintain detailed logs of data volume, load time and failure points for audit and debugging.

8.5. Apply Data Integration with Redundancy and Correlation analysis. (Consider an example).

→ Inside a healthcare system,

1) chr-patients: coming from the EHR system.

lab-results: coming from the LIS system.

Redundancy check and Removal.

Detecting duplicates

```
SELECT DISTINCT UPPER(patient-name), patient-id, dob  
FROM chr-patients;
```

```
SELECT patient-name, COUNT(*)
```

```
FROM chr-patients
```

```
GROUP BY patient-name
```

```
HAVING COUNT(*) > 1;
```

2) checking if age can be determined from DOB.

```
SELECT patient-id, DOB, patient-age,
```

```
DATE_DIFF(CURRENT_DATE(), DATE(DOB), YEAR) AS computed-age,
```

```
Age = DATE_DIFF(CURRENT_DATE(), DATE(DOB), YEAR) AS Is-age,
```

```
Correct FROM chr-patients;
```

If Is-age-correct is false, age is redundant/inconsistent.

Correlation.

Correlating Cholesterol vs age.

SELECT

e.patient_id,
e.patient_name,
l.test_result AS cholesterol-101

FROM

ehr_patients as e

JOIN

lis_results as l ON e.patient_id = l.patient_id

WHERE

l.test_name = 'cholesterol';

6. Analyze data reduction strategies: Attribute subset, selection, regression and log linear models.

⇒ Attribute subset selection: It is a dimensionality reduction method.

• Select a minimum set of features such that the probability distribution of different classes given the values for those features is close as possible to the original distribution given the values of all features reduce no. of patterns in the patterns, easier to understand.

• Regression and log-linear Models:

→ Linear Regression: Data modelled to fit a straight line.

• often use least-square method to fit the line

→ Multiple Regression: Allows a response variable Y to be modelled as a linear function of multidimensional feature vector.

→ log-linear Model: Approximates discrete multidimensional

22/25

Name: Gaigi Suphalankar

Roll No: B027

SAPID: 60019220125

Branch: CSE(CIB)

DE ASSIGNMENT 2

Q-1. Explain key engineering consideration for the ingestion phase.

⇒ ① Source variety:

- Different formats: Structured (tables), semi-structured (JSON, XML), unstructured (image, video, text)
- Different protocols: API calls, database connections, file transfers

② Data velocity

- Has fast data arriving (eg. daily batch vs real time system)
- Batch ingestion (periodic uploads) vs streaming ingestion (continuous) real time flow.

③ Data volume:

- How much data are you pulling? (MB, GB, TB - per day?)
- Systems need to handle large volumes without failure.

④ Fault tolerance and variability:

- What happens if ingestion fails? (eg network issue, partial loads)
- You must not lose data.

⑤ Data Quality & Validation

- Garbage in = garbage out.
- Validate data at ingestion point (eg. correct format)

Q-2. Discuss different ways to ingest data - direct database connection, change data capture and API's

⇒ ① Database connection: Direct database connections involve establishing a direct link between a data ingestion tool and a database. This method allows for immediate access to the data within the database, making it suitable for applications that require real time data access.

- JDBC (Java Database Connectivity): A standard API for Java application to connect to database.
- ODBC (Open database connectivity): A standard API for Java application to connect to database.
- Data-specific API's: Many databases offer their own API's (e.g. MySQL connector, postgresql) for direct connection.
- Data virtualization: Techniques like data virtualization allow connecting to multiple data sources through single interface.
- ② Change Data Capture: It is a software process that identifies and tracks changes to data in a database.
 - Audit columns: By using "LAST-UPDATED" or "DATE-MODIFIED" columns, by adding them if not in the application.
 - Step 1: Get the max value of both the target tables 'created time' and 'updated time' columns.
 - Step 2: Select the rows with 'created time' greater than (2) the target table's max 'created time'.
- Table deltas: You can use table delta or 'table-diff' utilities to compare the data in two tables for non convergence. There are several examples of SQL scripts that can find the difference of two tables.
- Trigger based CDC: Defining database triggers and creating your own change log in shadow tables. Triggers fire before or after INSERT, UPDATE or DELETE commands operating at the level, some users prefer this approach. Some databases even have native support for triggers.

- Q3: Explain data transformations as a part of pipeline with example.
- ⇒ ① Data cleaning: Fixes issues like missing values, duplicates and inconsistencies.

Example: Replacing ~~Attt~~ ^{null} user email addresses with a placeholder or removing duplicate transactions.

- ② Data Validation: Ensures data meets predefined rules and constraints.

Example: Checking that all customers follow a specific format and are unique.

- ③ Data Formatting: Converts data type and target system requirements.

Example: Standardization of date formats (DD/MM/YYYY) → (YYYY-MM-DD) and converting string to integers.

- ④ Data Enrichment: Add new, useful information from external sources or calculations.

Example: Appending demographic data based on ZIP codes or calculating customer lifetime value.

- ⑤ Data Aggregation: Combines data into summary metrics for analysis.

Example: Summing all purchases made by a customer in a month.

- ⑥ Data Normalization: Standard data for consistency across the dataset.

Example: Converting all currency values to users or standardize test cases.

Q.4. Discuss data preparation transformation with optimizing the file format, optimizing with data partitioning, data cleaning.

→ Optimizing the file format:

- Choose an efficient file format improves storage, usage, read/write speed and overall performance. Traditional formats like csv and json are simple but inefficient for large datasets analytics.

- Columnar formats like Parquet and ORC store data by column enabling faster query performance when accessing only selected columns.
- ~~Also~~ Avro is suitable for row-based processing and streaming data and it supports schema evolution.
- Optimizing with data partitioning.
- Partitioning breaks datasets into smaller segments. Helps engines like Spark, Hive read only relevant data, improving query importance.
- Example: A sales dataset partitioned by year and month allow January 2024 queries to access just that partition.
- Benefits of data partitioning:
 - Improve scalability
 - Improve Performance
 - Improve Security
 - Improve Availability.
- Data Cleansing
- Cleansing ensures that the data is accurate, consistent and usable. Common issues include missing values, duplicates, inconsistent formats, typo, etc.
- Cleansing Techniques:
 - 1) Handling missing data.
 - 2) Remove duplicates
 - 3) Type and range validation

Q.5. Explain in detail: Active security and the principle of least privilege.

- (i) Active security: Refers to the real time protection mechanisms that monitor, detect and respond to cybersecurity threats activity, rather than relying solely on predefined

rules or passive defenses.

Key features:

① → Real time monitoring: Continuously watches system activity, networks traffic and user behaviour.

② → Automated threat response: Quickly blocks, disables suspected malicious activity through automated system.

③ Principle of Least Privilege (PoLP): A fundamental cybersecurity concept where users and systems are granted the minimum level of access rights required to perform main functions.

Implementation:

- Assign roles with only necessary permissions.
- Regular audit user roles and access rights
- Use time bound access controls for temporary tasks.

Q.6. Explain in detail cataloguing your data to avoid data Swamp.

⇒ ① Data Swamp:

A data swamp is a poorly managed data value that lacks structure, metadata and governance, making it difficult to find trust and one data.

• Unlike a well organized data lake, a data swamp leads to data chaos where datasets are inaccessible, redundant, outdated and low quality.

② Importance of data Cataloguing:

• It is a process of creating an organized inventory of data assets including metadata, lineage and relationship between datasets.

• It acts like a library index, helping users discover, understand and use the right data efficiently.

⑤ Features of data cataloguing

- Metadata management
- Search and Discovery.
- Data lineage tracing.

⑥ How cataloguing prevents a data swamp.

- Improves visibility; Users can find and reuse datasets rather than duplicating efforts.
- Enables governance: Ensures policies, access controls and data standards are enforced.
- Enhance data quality: Promote documentation, validation and consistent use of datasets.
- Support Compliance: Helps organizations meet data protection and audit requirements.